

# ITS INFORMATION AND COMMUNICATIONS TECHNOLOGY Academy

**NOME MODULO:** *Sicurezza Informatica*

**UNITÀ DIDATTICA:** Sicurezza Informatica 2

**Dott. Ing. Giuseppe Placanica**

**Silicondev S.p.A.**

Anno Accademico 2022-2023



**NB.** Le presentazioni devono pervenire alla Segreteria Didattica ITS ([segreteria@its-ictacademy.com](mailto:segreteria@its-ictacademy.com)) possibilmente a inizio Modulo o al termine della singola Unità Didattica. I materiali didattici verranno condivisi con gli Allievi solo al termine del Modulo.

# Sicurezza Informatica – U2

## programma didattico

- Introduzione alla sicurezza informatica: minacce, attacchi, asset, requisiti funzionali sulla sicurezza, principi di progettazione di sistemi sicuri, superfici di attacco, alberi di attacco
- Crittografia (cenni): crittografia simmetrica, autenticazione funzioni hash sicure, crittografia a chiave pubblica, firma digitale, certificati di chiave pubblica
- Politiche di controllo degli accessi: principi fondamentali, controllo di accessi in Unix/Linux, politiche di accesso basate su ruoli e su attributi
- Malware e contromisure: virus, worm, trojan, email spam, ransomware, bot malevoli, zombie, phishing, keylogger, spyware, adware, backdoors, rootkit
- Sicurezza dei sistemi informatici: Attacchi (D)DOS e contromisure, riconoscimento delle intrusioni, prevenzione delle intrusioni (firewall)



# Sicurezza Informatica – U2

## programma didattico

- Sicurezza nel software (codice): stack overflow, gestione dell'input utente, interazioni tra programmi, gestione dell'output
- Sicurezza nei sistemi operativi Unix/Linux (pianificazione, hardening, manutenzione)
- Principi e tecniche di sicurezza dei database
- Sicurezza nei container (Docker, Kubernetes)
- Sicurezza nel dispiegamento di software su cloud:
  - software e banche di dati, Open Source, GNU General Public License (GPL), licenze Creative Commons
- Attività pratiche (Esercitazioni guidate; Esercitazioni libere; Simulazioni; Case study)



# Sicurezza Informatica – U2

## suddivisione programma didattico

### ❖ Prima lezione

- Introduzione alla sicurezza informatica: minacce, attacchi, asset, requisiti funzionali sulla sicurezza, principi di progettazione di sistemi sicuri, superfici di attacco, alberi di attacco
- Crittografia (cenni): crittografia simmetrica, autenticazione funzioni hash sicure, crittografia a chiave pubblica, firma digitale, certificati di chiave pubblica

### ❖ Seconda lezione

- Politiche di controllo degli accessi: principi fondamentali, controllo di accessi in Unix/Linux, politiche di accesso basate su ruoli e su attributi
- Malware e contromisure: virus, worm, trojan, email spam, ransomware, bot malevoli, zombie, phishing, keylogger, spyware, adware, backdoors, rootkit



# Sicurezza Informatica – U2

## suddivisione programma didattico

### ❖ Terza lezione

- Sicurezza dei sistemi informatici: Attacchi (D)DOS e contromisure, riconoscimento di intrusioni, prevenzione di intrusioni (firewall)
- Sicurezza nel software (codice): stack overflow, gestione dell'input utente, interazioni tra programmi, gestione dell'output

### ❖ Quarta lezione

- Sicurezza nei sistemi operativi Unix/Linux (pianificazione, hardening, manutenzione)
- Principi e tecniche di sicurezza dei database

### ❖ Quinta lezione:

- Sicurezza nei container (Docker, Kubernetes)
- Sicurezza nel dispiegamento di software su cloud:
  - software e banche di dati; Open Source, GNU General Public License (GPL), licenze Creative Commons)
  - Attività pratiche (Esercitazioni guidate; Esercitazioni libere; Simulazioni; Case study)



# Sicurezza Informatica – U2

## suddivisione programma didattico

### ❖ Sesta lezione

- Recap
- Esercitazioni
- Esame finale



# Sicurezza Informatica – U2

## I lezione



# Sicurezza Informatica – U2

## Sommario I lezione

- La Sicurezza Informatica
- Minacce Informatiche
- Attacchi Informatici
- Gli asset nella sicurezza informatica
- Requisiti funzionali sulla sicurezza
- Principi di progettazione di Sistemi sicuri





# Sicurezza Informatica – U2

## Sommario I lezione

- Superfici di attacco
- Alberi di attacco
- Crittografia
- Funzioni Hash
- Crittografia a chiave pubblica
- Firma digitale
- Certificati



# La Sicurezza Informatica 1/2

La sicurezza informatica è un campo che riguarda la protezione delle informazioni digitali da accessi non autorizzati, danni o furti. È una disciplina di fondamentale importanza nella società moderna, in cui la dipendenza dalle tecnologie digitali è sempre più diffusa.

La sicurezza informatica si occupa di **identificare e mitigare le minacce informatiche**, che possono includere **malware, phishing, attacchi di rete e furti di identità**. Le misure di sicurezza informatica includono l'utilizzo di password robuste, l'aggiornamento regolare dei software, l'uso della crittografia per proteggere i dati sensibili e il backup regolare dei dati importanti.

Oltre a proteggere gli asset digitali, la sicurezza informatica si occupa anche di garantire la **riservatezza, l'integrità e la disponibilità delle informazioni**. Ciò implica l'adozione di misure come il controllo degli accessi, la crittografia dei dati in transito e a riposo, e la verifica dell'identità degli utenti.



# La Sicurezza Informatica 2/2

La sicurezza informatica richiede una responsabilità condivisa tra individui, organizzazioni e governi.

Ogni persona deve essere consapevole delle minacce informatiche comuni e adottare pratiche di sicurezza come l'uso di password uniche e la condivisione responsabile delle informazioni.

Le organizzazioni devono implementare politiche e procedure di sicurezza informatica, formare il personale sull'importanza della sicurezza e utilizzare strumenti di sicurezza come firewall, antivirus e sistemi di rilevamento delle intrusioni.

I governi devono promuovere leggi e regolamenti sulla sicurezza informatica e collaborare con le organizzazioni per proteggere le infrastrutture critiche e le informazioni sensibili.



# La Sicurezza Informatica 3/3

## Summary:

In conclusione, la sicurezza informatica è fondamentale per **proteggere** le informazioni digitali e **preservare la privacy** e la fiducia nella società digitale.

È un campo in continua evoluzione, in cui la consapevolezza, l'educazione e la collaborazione sono cruciali per affrontare le minacce sempre più sofisticate che si presentano nel mondo digitale.



# Le minacce informatiche 1/3

Le minacce informatiche rappresentano potenziali eventi o situazioni che sfruttano le **vulnerabilità** presenti nei sistemi informatici per danneggiare, compromettere o rubare dati sensibili.

Queste minacce possono provenire da attori malevoli, come hacker o criminali informatici, che mirano a ottenere un vantaggio personale o causare danni finanziari o reputazionali alle organizzazioni o agli individui.

Le minacce informatiche possono assumere diverse forme, tra cui **malware**, **phishing**, **attacchi di rete**, **furti di identità** e **attacchi di forza bruta**.

- Il **malware**, come virus, worm o ransomware, è progettato per infettare e danneggiare i sistemi informatici, compromettendo la sicurezza dei dati e delle reti.



# Le minacce informatiche 2/3

- Il **phishing** è un tipo di truffa che mira a indurre gli utenti a condividere informazioni sensibili, come password o dati finanziari, attraverso l'uso di messaggi ingannevoli o siti web falsi.
- Gli **attacchi di rete**, come il (DoS), mirano a compromettere la sicurezza dei sistemi informatici attraverso la violazione delle reti e dei dispositivi collegati.
- I **furti di identità** coinvolgono il furto di informazioni personali, come nome, indirizzo, numero di previdenza sociale o dati finanziari, per scopi illeciti.
- Gli **attacchi di forza bruta** tentano di indovinare le password o le chiavi di accesso utilizzando tentativi ripetuti fino a trovare la combinazione corretta.



# Le minacce informatiche 3/3

Per proteggere i sistemi informatici dalle minacce, sono necessarie misure di sicurezza come l'utilizzo di **software antivirus** e **firewall**, l'adozione di **password complesse** e uniche per ogni applicazione, l'aggiornamento regolare del software e l'educazione degli utenti sulla sicurezza informatica. Le organizzazioni devono anche implementare politiche di sicurezza informatica, eseguire test di vulnerabilità e monitorare costantemente le attività sospette.

## Summary:

In sintesi, le **minacce informatiche** rappresentano **rischi potenziali** per la sicurezza dei sistemi informatici e dei dati sensibili. La consapevolezza, l'educazione e l'adozione di misure di sicurezza adeguate sono essenziali per mitigare queste minacce e proteggere le informazioni digitali.



# Attacchi informatici 1/3

Gli attacchi informatici sono **azioni intenzionali**, a differenza di una minaccia informatica che rappresenta il potenziale per un attacco, messe in atto da individui o gruppi malintenzionati al fine di **sfruttare le vulnerabilità** dei sistemi informatici al fine di compromettere la sicurezza delle informazioni.

Questi attacchi possono essere rivolti a organizzazioni, istituzioni o individui e possono avere conseguenze significative in termini di danni finanziari, perdita di dati sensibili o compromissione della privacy.

Gli attacchi informatici possono assumere diverse forme e utilizzare diverse tecniche. Alcuni esempi comuni includono:

- **Attacchi di phishing:** gli attaccanti inviano messaggi di posta elettronica o messaggi istantanei ingannevoli che sembrano provenire da fonti affidabili, al fine di indurre le persone a rivelare informazioni personali o accedere a siti web dannosi.





# Attacchi informatici 2/3

- **Attacchi di malware:** gli attaccanti utilizzano software dannosi, come virus, worm o trojan, per infettare i sistemi informatici. Il malware può consentire agli attaccanti di controllare il sistema, rubare dati o danneggiare i file.
- **Attacchi di denial of service (DoS):** gli attaccanti sovraccaricano intenzionalmente un sistema o una rete con un'eccessiva quantità di traffico al fine di renderlo inutilizzabile per gli utenti legittimi.
- **Attacchi di brute force:** gli attaccanti cercano di violare la sicurezza di un sistema o di un account tentando ripetutamente diverse combinazioni di password finché non trovano quella corretta.
- **Attacchi di spoofing:** gli attaccanti fingono di essere qualcun altro, come un sito web legittimo o un indirizzo email, al fine di ingannare gli utenti e ottenere accesso o informazioni riservate. Esempio: email/sms truffa.



# Attacchi informatici 3/3

- **Attacchi di ingegneria sociale:** gli attaccanti manipolano le persone utilizzando tattiche psicologiche per indurle a rivelare informazioni confidenziali o compiere azioni dannose.

Per proteggersi dagli attacchi informatici, è importante adottare misure di sicurezza adeguate, come l'uso di software antivirus e firewall, l'aggiornamento regolare del software, l'implementazione di politiche di sicurezza informatica e la formazione del personale per riconoscere e rispondere agli attacchi.

## Summary:

In conclusione, **gli attacchi informatici** rappresentano una **minaccia significativa** per la sicurezza delle **informazioni** e possono avere conseguenze dannose per le **organizzazioni** e gli **individui**. La prevenzione, la consapevolezza e la prontezza nella risposta agli attacchi sono elementi chiave per proteggere i sistemi informatici e preservare la sicurezza delle informazioni.



# Esempi di attacchi informatici 1/2

Gli attacchi informatici possono assumere diverse forme e utilizzare varie tecniche per compromettere la sicurezza dei sistemi e delle informazioni. Di seguito sono riportati alcuni esempi comuni di attacchi informatici:

Attacchi di **forza bruta**: in questo tipo di attacco, gli aggressori tentano di **indovinare le credenziali** di accesso (come username e password) utilizzando un numero elevato di combinazioni possibili. Questo attacco può essere utilizzato per violare la sicurezza di account utente o di sistemi protetti da password deboli o facilmente prevedibili.

Attacchi **DDoS** (Distributed Denial of Service): in un attacco DDoS, gli aggressori utilizzano una rete di dispositivi infetti (botnet) per inviare una quantità massiccia di traffico al sistema target, sovraccaricandolo e rendendolo inaccessibile agli utenti legittimi. Questo tipo di attacco mira a interrompere o destabilizzare i servizi online.



# Esempi di attacchi informatici 2/2

**Furto di credenziali:** questo attacco coinvolge l'intercettazione o il furto delle credenziali di accesso di un utente, come username e password. Gli aggressori possono utilizzare varie tecniche, come il **phishing**, per ottenere le credenziali e accedere ai sistemi o alle informazioni riservate dell'utente.

**Exploit** delle vulnerabilità del software: gli aggressori sfruttano le **vulnerabilità** presenti nel software o nel sistema operativo per ottenere accesso non autorizzato o eseguire codice dannoso. Questi exploit possono essere utilizzati per compromettere la sicurezza del sistema, rubare informazioni o installare malware. (Es. reverse shell)

Altri esempi di attacchi informatici includono attacchi di ingegneria sociale, in cui gli aggressori manipolano le persone per ottenere informazioni sensibili o compiere azioni dannose, e attacchi di spoofing, in cui gli aggressori fingono di essere qualcun altro (come un sito web o un indirizzo email) per ingannare gli utenti.



# Gli Asset di sicurezza 1/3

Gli asset rappresentano **risorse critiche** all'interno di un sistema informatico che devono essere adeguatamente protette per garantire la sicurezza delle informazioni e la continuità delle operazioni. Gli asset possono assumere diverse forme, tra cui **dati, software, hardware, reti** e infrastrutture.

I **dati** sono uno degli asset più importanti e includono informazioni sensibili, come dati personali, informazioni aziendali riservate, segreti commerciali o dati finanziari. La protezione dei dati è essenziale per prevenire violazioni della privacy, frodi o perdite di informazioni sensibili.

Il **software** è un altro asset critico che richiede protezione... A volte è proprio la porta d'ingresso. Questo include sistemi operativi, applicazioni, programmi e codici sorgente. La sicurezza del software è fondamentale per prevenire vulnerabilità e falle che potrebbero essere sfruttate dagli attaccanti per compromettere il sistema o rubare informazioni.



# Gli Asset di sicurezza 2/3

L'**hardware** rappresenta gli elementi fisici del sistema informatico, come server, computer, dispositivi di rete e dispositivi di archiviazione. La protezione dell'hardware implica l'adozione di misure fisiche, come la sicurezza dei locali e la gestione degli accessi, per prevenire il furto, l'alterazione o il danneggiamento degli apparati.

Le **reti** e le **infrastrutture** sono anch'esse asset critici che richiedono protezione. Questi includono le reti locali (LAN), le reti senza fili (Wi-Fi), le reti estese (WAN), i router, i firewall e i dispositivi di networking. La sicurezza delle reti implica l'implementazione di controlli di accesso, la crittografia delle comunicazioni e la rilevazione delle intrusioni per proteggere le informazioni che transitano sulla rete.

Attenzione alle Wi-Fi aperte!



# Gli Asset di sicurezza 3/3

**Proteggere gli asset** significa implementare una combinazione di misure tecniche, come l'utilizzo di **software di sicurezza**, la **crittografia** dei dati, il monitoraggio delle attività sospette e l'applicazione di patch di sicurezza regolari. Inoltre, è importante adottare politiche e procedure di sicurezza solide, fornire formazione agli utenti e stabilire controlli di accesso appropriati per garantire che gli asset siano **accessibili solo a persone autorizzate**.

## Summary:

In sintesi, gli asset rappresentano le **risorse critiche** all'interno di un sistema informatico che **devono essere protette**. La protezione degli asset è fondamentale per garantire la sicurezza delle informazioni e la **continuità delle operazioni**. Ciò richiede l'implementazione di misure tecniche e organizzative per prevenire violazioni della sicurezza e mitigare i rischi associati.



# I requisiti funzionali 1/3

I requisiti funzionali sulla sicurezza **definiscono** le funzionalità e le **misure** di sicurezza che un sistema informatico deve avere al fine di **proteggere gli asset critici**. Questi requisiti si concentrano sulle azioni e sulle capacità che il sistema deve essere in grado di svolgere per garantire un adeguato livello di sicurezza.

I requisiti funzionali sulla sicurezza possono includere **diverse componenti**, tra cui:

- **Autenticazione:** il sistema deve essere in grado di autenticare in modo affidabile l'identità degli utenti o dei dispositivi che accedono al sistema. Ciò può richiedere l'utilizzo di password, certificati digitali, biometria o altre forme di autenticazione.
- **Autorizzazione:** il sistema deve consentire solo agli utenti autorizzati di accedere alle risorse o alle funzionalità specifiche del sistema. Ciò può essere implementato attraverso l'assegnazione di privilegi o ruoli appropriati agli utenti e il controllo degli accessi in base alle autorizzazioni definite.





# I requisiti funzionali 2/3

- **Controllo degli accessi:** il sistema deve essere in grado di gestire e controllare l'accesso alle risorse in modo granulare. Ciò può includere la definizione di politiche di accesso, la gestione delle credenziali, l'implementazione di controlli di sicurezza come firewall o VPN e l'audit dei tentativi di accesso non autorizzati.
- **Crittografia:** il sistema deve supportare l'utilizzo di tecniche di crittografia per proteggere i dati in transito e a riposo. Ciò può includere la crittografia dei dati sensibili, la crittografia delle comunicazioni su reti non sicure e la gestione sicura delle chiavi crittografiche.
- **Monitoraggio e rilevamento delle intrusioni:** il sistema deve essere in grado di rilevare, monitorare e registrare le attività sospette o anomale. Ciò può essere realizzato attraverso l'implementazione di strumenti di monitoraggio dei log, sistemi di rilevamento delle intrusioni (IDS) o sistemi di prevenzione delle intrusioni (IPS).



# I requisiti funzionali 3/3

- **Gestione delle vulnerabilità:** il sistema deve essere in grado di identificare e gestire le vulnerabilità del software o delle configurazioni che potrebbero essere sfruttate dagli aggressori. Ciò può richiedere l'implementazione di procedure per l'aggiornamento continuo dei sistemi. (CVE)
- **Continuità del servizio:** il sistema deve essere progettato per garantire la continuità delle operazioni anche in caso di eventi imprevisti o di attacchi. Ciò può includere la pianificazione di backup dei dati, la creazione di piani di ripristino di emergenza e l'adozione di misure per garantire la disponibilità e l'integrità dei servizi.

I **requisiti funzionali** sono essenziali per garantire che un sistema informatico **sia progettato e implementato** correttamente per **proteggere gli asset critici**. Essi devono essere definiti e integrati nel processo di sviluppo del sistema, garantendo una progettazione fin dall'inizio.



# Progettazione Sicura:

## Limitare l'accesso

1/11

Uno dei principi fondamentali nella progettazione di sistemi sicuri è quello di **limitare l'accesso solo alle risorse necessarie** per svolgere determinate attività. Questo principio, noto anche come "**modello del privilegio minimo**" o "**accesso basato su ruoli**", si basa sul concetto di garantire che gli utenti o i processi abbiano solo i privilegi necessari per eseguire le loro funzioni specifiche e non di più.

Limitare l'accesso alle risorse necessarie riduce il rischio di violazioni della sicurezza dovute ad accessi non autorizzati o ad abusi di privilegi. Invece di concedere agli utenti accesso illimitato a tutte le risorse del sistema, si adotta un approccio di "**principio del bisogno di conoscenza**", in cui gli utenti hanno solo l'accesso alle risorse e ai dati che sono rilevanti per il loro lavoro.



# Progettazione Sicura:

## Limitare l'accesso

2/11

Questo principio può essere implementato attraverso la **definizione di ruoli e privilegi** all'interno del sistema. Ogni ruolo è associato a un insieme specifico di autorizzazioni e accessi consentiti. Gli utenti vengono assegnati a uno o più ruoli in base alle loro **responsabilità** e funzioni all'interno dell'organizzazione. In questo modo, gli utenti hanno solo accesso alle risorse e alle funzionalità che sono strettamente necessarie per svolgere il loro lavoro.

Limitare i privilegi e i permessi degli utenti aiuta a mitigare il potenziale impatto di un'eventuale compromissione del sistema o delle credenziali degli utenti.



# Progettazione Sicura:

## Limitare l'accesso

3/11

Oltre a limitare l'accesso alle risorse necessarie, ci sono altri principi di progettazione di sistemi sicuri, come l'utilizzo di tecniche di **crittografia** per proteggere i dati sensibili, l'implementazione di meccanismi di autenticazione forte, l'adozione di best practice per la **gestione delle password** e l'uso di controlli di sicurezza come **firewall** e **sistemi di rilevamento delle intrusioni**.

In conclusione, il principio di limitare l'accesso solo alle risorse necessarie mira a garantire che gli utenti e i processi abbiano solo i privilegi e gli **accessi appropriati** per svolgere le loro funzioni specifiche, riducendo così il rischio di violazioni della sicurezza e garantendo la protezione delle informazioni sensibili.



# Progettazione Sicura:

## Privilegio minimo

4/11

Il principio di progettazione sicura del "minimo privilegio" è un concetto fondamentale che mira a limitare i privilegi di accesso e le autorizzazioni solo al livello strettamente necessario per svolgere le attività specifiche. In pratica, significa che agli utenti, ai processi o ai servizi vengono assegnati solo i **privilegi minimi necessari** per svolgere il loro lavoro o le loro funzioni, evitando di concedere privilegi o accessi più elevati di quanto richiesto.

Il principio del minimo privilegio contribuisce a ridurre il rischio di abusi o utilizzi impropri delle autorizzazioni. **Limitando l'accesso solo alle risorse necessarie, si riduce la superficie di attacco** e si impedisce agli utenti o ai processi di accedere o modificare informazioni o risorse a cui non dovrebbero avere accesso. Inoltre, in caso di compromissione di un account o di un processo, l'impatto potenziale è limitato, poiché l'attaccante avrà solo accesso a un insieme ridotto di risorse.



# Progettazione Sicura:

## Privilegio minimo

5/11

Questo principio può essere applicato in diversi contesti. Ad esempio, nell'ambito delle autorizzazioni degli utenti, si assegnano solo i privilegi di accesso necessari alle risorse pertinenti per il loro ruolo o le loro responsabilità. Inoltre, nei sistemi operativi o nelle applicazioni, i processi o i servizi vengono eseguiti con privilegi minimi, senza concedere autorizzazioni di amministratore o di root a meno che non siano strettamente necessarie.

L'applicazione del principio del minimo privilegio richiede un'**analisi** attenta **delle autorizzazioni richieste per ogni utente, processo** o servizio, insieme a una corretta pianificazione delle politiche di accesso e dei meccanismi di controllo. È importante valutare e rivedere **periodicamente** le autorizzazioni assegnate, garantendo che siano allineate alle effettive esigenze operative e rimuovendo o riducendo i privilegi non più necessari.



# Progettazione Sicura:

## Privilegio minimo

6/11

Applicando questo principio, si riduce il rischio di abusi o di accessi non autorizzati, contribuendo a garantire un ambiente sicuro e protetto per i sistemi informatici e le risorse.

Nella progettazione sicura, "limitare l'accesso" e "principio del privilegio minimo" sono due concetti distinti ma correlati:

- **Limitare l'accesso** si riferisce al **controllo generale degli accessi**, solo le persone autorizzate possono accedere a un sistema o a una risorsa.
- **Il privilegio minimo** si riferisce invece **alla concessione di autorizzazioni** solo per le attività specifiche richieste, evitando di assegnare privilegi eccessivi o non necessari agli utenti.

Entrambi i principi sono fondamentali per garantire la sicurezza dei sistemi informatici e la protezione delle informazioni sensibili.





# Progettazione Sicura: Difesa in profondità

7/11

Il principio di progettazione sicura della "difesa in profondità" è un approccio strategico che prevede la creazione di una **serie di strati di sicurezza** per proteggere un sistema informatico da diverse minacce. Invece di affidarsi a una singola misura di sicurezza, la difesa in profondità si basa sulla **combinazione di diverse strategie** e controlli di sicurezza per mitigare i rischi e garantire un ambiente sicuro.

L'idea principale è quella di prevedere misure di sicurezza in modo **gerarchico**, con strati multipli che agiscono come barriere di contenimento e operano sinergicamente per proteggere il sistema.

Gli strati di sicurezza possono includere:

- Protezione fisica: edifici, server e device
- Firewall



# Progettazione Sicura: Difesa in profondità

8/11

- Controllo degli accessi: password complesse, autenticazione a due fattori, certificati digitali o token di sicurezza
- Crittografia: algoritmi crittografici sui dati
- Sistemi di rilevamento delle intrusioni
- Backup e ripristino

Questo principio si basa sul fatto che **nessuna singola misura di sicurezza è completamente efficace nel proteggere un sistema**. Integrando diversi strati di sicurezza, si crea un ambiente resistente alle minacce informatiche, in cui gli attaccanti devono superare multiple barriere per ottenere l'accesso non autorizzato.

Questo approccio aumenta la resilienza del sistema e riduce il rischio di violazioni della sicurezza, consentendo una risposta tempestiva alle minacce e la protezione degli asset critici.



# Progettazione Sicura: 9/11

## Separazione dei compiti

Il principio di progettazione sicura della "separazione dei compiti" si riferisce alla pratica **di suddividere le responsabilità e le autorizzazioni** all'interno di un sistema informatico tra più entità o utenti. L'obiettivo principale è **impedire** che una singola entità o utente abbia **accesso completo e illimitato** a tutte le funzioni o risorse critiche del sistema (es: attacco nucleare).

La separazione dei compiti aiuta a mitigare il rischio di abusi di privilegi o di attacchi interni, in quanto richiede la collaborazione di più entità per completare determinate attività.

In questo modo, si limita la possibilità che un singolo individuo possa sfruttare il proprio accesso per scopi dannosi o fraudolenti.



# Progettazione Sicura: 10/11

## Separazione dei compiti

La separazione dei compiti può essere implementata attraverso l'assegnazione di ruoli e responsabilità chiaramente definiti a diversi utenti o gruppi all'interno del sistema.

Ogni ruolo ha autorizzazioni specifiche per svolgere determinate funzioni, e l'accesso a risorse critiche è limitato solo ai ruoli necessari per eseguire quelle attività.

Ciò significa che anche se un utente dovesse ottenere l'accesso a un particolare account o ruolo, **non** avrebbe automaticamente **accesso a tutte le funzionalità** o risorse del sistema.



# Progettazione Sicura: 11/11

## Separazione dei compiti

L'implementazione della separazione dei compiti richiede una pianificazione oculata delle autorizzazioni e un'analisi delle responsabilità dei diversi utenti o entità coinvolte. È importante che le autorizzazioni siano assegnate in modo coerente con le esigenze operative e che vengano applicati meccanismi di **controllo** e revisione per garantire che le autorizzazioni siano mantenute in modo appropriato **nel tempo**.

In estrema sintesi, questo principio mira a **limitare l'accesso completo e illimitato** a una singola entità o utente, suddividendo le responsabilità e le autorizzazioni tra più entità. Questo approccio contribuisce a mitigare il rischio di abusi di privilegi e attacchi interni, proteggendo le risorse critiche del sistema e promuovendo un ambiente sicuro.



# Superfici di attacco 1/2

Nella progettazione sicura, una "superficie di attacco" rappresenta l'insieme di **punti di ingresso** o **vulnerabilità** che possono essere sfruttati da un attaccante per compromettere la sicurezza di un sistema informatico. Questi punti di ingresso possono essere costituiti da componenti hardware, software, reti o errori umani che possono essere sfruttati per ottenere accesso non autorizzato o causare danni al sistema.

Identificare e comprendere le superfici di attacco è un aspetto critico della progettazione sicura, poiché consente di prendere le misure necessarie per mitigare i rischi e proteggere il sistema. Alcuni esempi comuni di superfici di attacco includono:

- **Interfacce di rete:** Le interfacce di rete, come porte Ethernet o Wi-Fi, forniscono un punto di ingresso nel sistema che può essere utilizzato per attacchi di rete, come l'intercettazione del traffico o l'iniezione di pacchetti malevoli.
- **Applicazioni Web:** Le applicazioni Web possono presentare vulnerabilità come errori di programmazione, falle di sicurezza o vulnerabilità di scripting lato server o lato client, che possono essere sfruttate per ottenere accesso non autorizzato o compromettere i dati.



# Superfici di attacco 2/2

- **Protocolli di comunicazione:** I protocolli di comunicazione, come HTTP, FTP o SMTP, possono essere sfruttati per attacchi come il furto di dati, l'intercettazione delle comunicazioni o l'iniezione di codice malevolo.
- **Dispositivi di input/output:** I dispositivi di input/output, come tastiere, mouse o periferiche di archiviazione esterne, possono presentare vulnerabilità fisiche o logiche che possono essere sfruttate per eseguire attacchi come l'intercettazione delle informazioni o l'infezione da malware.
- **Errori umani:** Gli errori umani, come l'uso di password deboli, la mancata applicazione di patch di sicurezza o la condivisione non sicura di informazioni, possono costituire una superficie di attacco, poiché gli attaccanti possono sfruttare queste debolezze per ottenere accesso non autorizzato.



# Esempi di Superfici di attacco

Oltre a tutte le superfici di attacco comuni e stra conosciute, fate attenzione a:

- **Dispositivi IoT** non sicuri: Gli oggetti di Internet of Things (IoT) possono essere vulnerabili agli attacchi se non sono adeguatamente protetti. Ad esempio, una telecamera di sicurezza o un dispositivo di controllo domotico non sicuro può essere sfruttato dagli attaccanti per ottenere accesso alla rete o compromettere la privacy.
- Furti di credenziali: Gli attaccanti possono cercare di ottenere credenziali utente, come nomi utente e password, attraverso tecniche di **phishing**, **keylogging** o attacchi di **social engineering**. Le credenziali rubate possono quindi essere utilizzate per ottenere accesso non autorizzato ai sistemi o alle informazioni sensibili.





# Alberi di attacco 1/3

Gli alberi di attacco sono uno strumento utile per analizzare le vulnerabilità di un sistema, identificare i percorsi di attacco potenziali e sviluppare strategie di mitigazione per proteggere il sistema da tali attacchi.

Vediamo come possono essere utilizzati in dettaglio:

- **Analisi delle vulnerabilità:** Gli alberi di attacco consentono agli esperti di sicurezza di esaminare in dettaglio le possibili vulnerabilità di un sistema. Ogni **nodo** dell'albero rappresenta una possibile **debolezza** o punto di accesso che potrebbe essere sfruttato dagli attaccanti. Attraverso l'analisi delle diverse diramazioni dell'albero, è possibile identificare le vulnerabilità specifiche e valutarne l'impatto potenziale sugli obiettivi di sicurezza del sistema.



# Alberi di attacco 2/3

- **Identificazione dei percorsi di attacco:** Gli alberi di attacco aiutano a visualizzare i possibili percorsi che un attaccante potrebbe seguire per raggiungere i suoi obiettivi. Questo include le diverse **fasi dell'attacco**, come l'ottenimento di accesso iniziale, l'escalation dei privilegi, l'esplorazione del sistema, l'acquisizione di dati sensibili, ecc. Attraverso l'analisi dell'albero, è possibile identificare i punti critici in cui gli attaccanti potrebbero sfruttare le vulnerabilità del sistema.
- **Sviluppo di strategie di mitigazione:** Gli alberi di attacco forniscono una panoramica chiara delle possibili vie di attacco, consentendo agli esperti di sicurezza di sviluppare strategie di mitigazione appropriate. Attraverso l'analisi dell'albero, è possibile identificare le contromisure necessarie per ridurre le vulnerabilità e interrompere i percorsi di attacco.



# Alberi di attacco 3/3

## Summary:

In sintesi, l'utilizzo degli alberi di attacco nell'analisi delle vulnerabilità e nello sviluppo di strategie di mitigazione consente agli esperti di sicurezza di **identificare** e **comprendere** i percorsi di attacco potenziali, focalizzarsi sulle **vulnerabilità** critiche e adottare **misure preventive** per proteggere il sistema da attacchi informatici.

Inoltre, gli alberi di attacco possono essere utilizzati per condurre simulazioni di attacco e test di penetrazione per valutare l'efficacia delle contromisure e identificare eventuali punti deboli rimanenti.



# Test

1. Cos'è la sicurezza informatica?
  - a. Protezione fisica dei computer.
  - b. Protezione delle informazioni e dei sistemi informatici da minacce e attacchi.
  - c. Creazione di password complesse.
  - d. Installazione di software antivirus.
2. Quali sono le minacce informatiche comuni?
  - a. Malware.
  - b. Phishing.
  - c. Attacchi di rete.
  - d. Tutte le precedenti.
3. Cosa significa DoS?
  - a. Denial of Service.
  - b. Data Destruction on Server.
  - c. Distributed Data Security.
  - d. Database Development and Support.



# Test

4. Cos'è l'ingegneria sociale?
  - a. Un approccio per ingegnerizzare software complessi.
  - b. L'utilizzo della sicurezza informatica per scopi di ingegneria.
  - c. Manipolazione psicologica per ottenere informazioni riservate.
  - d. Sviluppo di algoritmi di crittografia avanzati.
  
5. Quali sono i principali obiettivi della sicurezza informatica?
  - a. Riservatezza, integrità e disponibilità dei dati.
  - b. Aumento delle prestazioni del sistema informatico.
  - c. Riduzione dei costi operativi.
  - d. Semplicità dell'interfaccia utente.



# Crittografia

La crittografia è una disciplina che si occupa di **proteggere e rendere sicure le informazioni** tramite l'utilizzo di **algoritmi matematici**. Consiste nell'elaborare un dato in modo tale che possa essere letto solo da chi è autorizzato ad accedere a esso, rendendo incomprensibile il contenuto originale per chiunque tenti di intercettarlo senza autorizzazione.

Don't invent your own crypto-  
but use well-established ones

Applied Cryptography

secret      writing

Nel contesto della sicurezza informatica, la crittografia viene utilizzata per **garantire la riservatezza, l'integrità e l'autenticità** delle informazioni. I dati vengono crittografati (o "cifrati") utilizzando una chiave segreta o una coppia di chiavi pubblica-privata.

La chiave di crittografia è un parametro critico!



# Crittografia Simmetrica 1/9

La crittografia simmetrica è una **tecnica di crittografia** in cui si utilizza la **stessa chiave segreta** per cifrare e decifrare i dati.

La chiave segreta, anche chiamata chiave di cifratura, è condivisa tra il mittente e il destinatario del messaggio... (Problema!)

Nella crittografia simmetrica, il processo di cifratura trasforma i dati in un formato illeggibile chiamato testo cifrato, utilizzando la chiave segreta. Questo processo rende i dati incomprensibili a chiunque non abbia la chiave segreta corretta per decifrare il testo cifrato.



# Crittografia Simmetrica 2/9

## Mono-alphabetic substitution

| Cleartext<br>alphabet | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z |
|-----------------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Key                   | J | U | L | I | S | C | A | E | R | T | V | W | X | Y | Z | B | D | F | G | H | K | M | N | O | P | Q |

**P** = "TWO HOUSEHOLDS, BOTH ALIKE IN DIGNITY,  
IN FAIR VERONA, WHERE WE LAY OUR SCENE"  
(“Romeo and Juliet”, Shakespeare)





# Crittografia Simmetrica 3/9

Una volta che il testo cifrato viene trasmesso al destinatario, quest'ultimo utilizza la stessa chiave segreta per decifrare il messaggio e ripristinare i dati originali.

Il processo di decifratura utilizza l'algoritmo di decifratura inverso rispetto all'algoritmo di cifratura utilizzato dal mittente.

La crittografia simmetrica è **veloce ed efficiente** poiché richiede meno risorse computazionali rispetto alla crittografia asimmetrica (a chiave pubblica).

Tuttavia, un aspetto critico nella crittografia simmetrica è la **gestione sicura della chiave segreta**, poiché entrambi il mittente e il destinatario devono condividere la stessa chiave segreta in modo che possano cifrare e decifrare i messaggi correttamente.



# Crittografia Simmetrica 4/9

Seguono alcuni esempi di crittografia simmetrica ampiamente utilizzati:

- Data Encryption Standard (DES)
- Advanced Encryption Standard (AES)
- Triple Data Encryption Standard (3DES)

Questi algoritmi sono stati sviluppati per fornire un alto livello di sicurezza e sono generalmente utilizzati in applicazioni come la crittografia dei dati, la protezione delle comunicazioni e la sicurezza delle informazioni.



# Crittografia Simmetrica 5/9

Il funzionamento della crittografia simmetrica può essere suddiviso in tre fasi: generazione della chiave, cifratura e decifratura.

1. **Generazione della chiave:** Inizialmente, è necessario generare una chiave segreta, che può essere un **valore casuale** di una lunghezza appropriata. La **sicurezza** della crittografia dipende in gran parte **dalla complessità della chiave** utilizzata.
2. **Cifratura:** Nel processo di cifratura, i dati originali (testo in chiaro) vengono combinati con la chiave segreta utilizzando un algoritmo di cifratura. Questo algoritmo esegue una serie di operazioni matematiche per convertire i dati in chiaro in un formato illeggibile noto come testo cifrato. Solo chi possiede la chiave segreta corretta sarà in grado di decifrare il testo cifrato per ripristinare i dati originali.



# Crittografia Simmetrica 6/9

- 3. Decifratura:** Per decifrare il testo cifrato e recuperare i dati originali, il destinatario deve utilizzare la stessa chiave segreta utilizzata durante la cifratura. L'algoritmo di decifratura esegue l'inverso delle operazioni di cifratura per ripristinare il testo in chiaro dai dati cifrati.

Durante il processo di crittografia simmetrica, sia il mittente che il destinatario devono condividere in modo sicuro la chiave segreta, poiché la sicurezza del sistema dipende dalla segretezza della chiave.

Questo è un punto critico nella sicurezza della crittografia simmetrica, poiché la gestione sicura delle chiavi richiede protezione da accessi non autorizzati.



# Crittografia Simmetrica 7/9

L'utilizzo della crittografia simmetrica è ampiamente diffuso in vari scenari, come la **protezione dei dati** su supporti di archiviazione, la sicurezza delle comunicazioni e la crittografia dei file.

È importante notare che, sebbene la crittografia simmetrica sia efficiente e veloce, il suo **principale svantaggio** è la necessità di **condividere la chiave segreta** in modo sicuro tra i partecipanti, il che può essere problematico in alcuni scenari di comunicazione.

Per superare questa limitazione, viene spesso utilizzata una combinazione di crittografia simmetrica e crittografia asimmetrica (a chiave pubblica) in sistemi crittografici più complessi.



# Crittografia Simmetrica 8/9

Come anticipato, due esempi di algoritmi di crittografia simmetrica ampiamente utilizzati sono DES (Data Encryption Standard) e AES (Advanced Encryption Standard):

DES (Data Encryption Standard): DES è un algoritmo di crittografia a blocchi sviluppato negli anni '70.

Utilizza una chiave di 56 bit per cifrare i dati in blocchi di 64 bit. DES esegue una serie di trasposizioni, sostituzioni e permutazioni per cifrare e decifrare i dati.

Tuttavia, negli anni successivi, la lunghezza della **chiave di 56 bit** è stata considerata troppo corta per fornire una sicurezza adeguata contro gli attacchi di forza bruta. Pertanto, DES è stato gradualmente sostituito da algoritmi più sicuri come AES.



# Crittografia Simmetrica 9/9

AES (Advanced Encryption Standard): AES è un algoritmo di crittografia a blocchi utilizzato come standard nella crittografia simmetrica a livello mondiale.

È stato introdotto nel 2001 come successore di DES ed è diventato l'algoritmo di crittografia più comunemente utilizzato.

AES supporta lunghezze di chiave di 128, 192 e 256 bit ed esegue operazioni di sostituzione, permutazione e trasposizione per cifrare e decifrare i dati. AES è noto per la sua sicurezza e prestazioni elevate ed è ampiamente utilizzato in applicazioni che richiedono una forte crittografia, come la protezione dei dati sensibili, le comunicazioni sicure e la crittografia dei file.

Si è preferito l'AES al DES per la lunghezza delle chiavi e per le sue proprietà crittografiche avanzate.



# Funzioni Hash 1/2

Una funzione hash è una funzione matematica che prende in input dati di qualsiasi dimensione e restituisce un valore hash di lunghezza fissa. L'obiettivo principale di una funzione hash è quello di generare un "**riassunto**" univoco e identificativo dei dati in input.

Questo riassunto, chiamato hash, è un valore numerico o una sequenza di caratteri che rappresenta in modo univoco i dati originali.

Le funzioni hash sono progettate per essere veloci da calcolare e restituire un hash unico per ogni input diverso (non sempre vero). Quindi, anche una piccola modifica ai dati in input dovrebbe produrre un hash completamente diverso. Questa proprietà, chiamata "**proprietà di resistenza alla collisione**", rende le funzioni hash utili **per verificare l'integrità dei dati**.





# Funzioni Hash 2/2

Le funzioni hash sono ampiamente utilizzate in vari ambiti:

- **Integrità dei dati:** Le funzioni hash vengono utilizzate per verificare se i dati sono stati alterati o corrotti.
- **Crittografia delle password:** Le funzioni hash sono utilizzate per memorizzare le password in modo sicuro e cifrate.
- **Firma digitale:** Le funzioni hash vengono utilizzate per creare firme digitali.

Alcuni esempi di funzioni hash sicure ampiamente utilizzate includono SHA-256, SHA-3, MD5 e SHA-1 (quest'ultima ormai considerata non sicura e non raccomandata per nuove applicazioni a causa di vulnerabilità note).



# Crittografia a chiave pubblica 1/4

La crittografia a chiave pubblica, nota anche come crittografia asimmetrica, è un metodo crittografico che coinvolge **l'uso di una coppia di chiavi**: una chiave pubblica e una chiave privata.

Questo tipo di crittografia si basa su algoritmi matematici complessi che consentono a un **mittente** di cifrare i dati utilizzando la **chiave pubblica del destinatario** e al **destinatario** di decifrare i dati utilizzando la **sua chiave privata** corrispondente.

Nella crittografia a chiave pubblica, la chiave pubblica viene diffusa senza vincoli e può essere conosciuta da tutti. La chiave privata, al contrario, viene mantenuta segretamente dal proprietario e non viene divulgata. La **coppia di chiavi è matematicamente correlata** in modo tale che ciò che viene crittografato con la chiave pubblica possa essere decifrato solo con la chiave privata corrispondente e viceversa.



# Crittografia a chiave pubblica 2/4

Il funzionamento della crittografia a chiave pubblica o asimmetrica può essere descritto brevemente descrivendo le seguenti fasi:

## Cifratura:

- Il destinatario genera una coppia di chiavi: una chiave pubblica e una chiave privata.
- La chiave pubblica viene diffusa ampiamente e resa disponibile a chiunque voglia inviare dati crittografati al destinatario.
- Un mittente che desidera inviare dati al destinatario utilizza la chiave pubblica del destinatario per crittografare i dati. Solo il destinatario, che possiede la corrispondente chiave privata, può decifrare i dati.



# Crittografia a chiave pubblica 3/4

## Decifratura:

- Il destinatario riceve i dati crittografati e utilizza la sua chiave privata per decifrare i dati e ripristinarli al loro stato originale.
- La chiave privata è mantenuta segretamente dal destinatario e non è conosciuta da nessun altro, garantendo la confidenzialità delle informazioni.

La crittografia a chiave pubblica offre diversi vantaggi rispetto alla crittografia simmetrica.

La crittografia a chiave pubblica consente una **comunicazione sicura** anche in scenari in cui mittente e destinatario non hanno avuto un precedente scambio di chiavi. Inoltre, viene utilizzata per scopi come l'autenticazione, la firma digitale e lo scambio sicuro delle chiavi simmetriche.



# Crittografia a chiave pubblica 4/4

Alcuni algoritmi di crittografia a chiave pubblica noti includono:

- RSA (Rivest-Shamir-Adleman)
- Diffie-Hellman
- ECC (Elliptic Curve Cryptography)

Questi algoritmi offrono diverse proprietà matematiche e livelli di sicurezza, ma tutti si basano sul concetto di utilizzo di una coppia di chiavi correlati per cifrare e decifrare i dati in modo sicuro.



# Firma digitale 1/5

La firma digitale è un meccanismo crittografico utilizzato per garantire **l'autenticità**, **l'integrità** e la **non ripudiabilità** di un messaggio o di un documento elettronico.

Si basa sulla crittografia a chiave pubblica e permette a un **mittente** di firmare digitalmente un documento utilizzando la **propria chiave privata**, mentre il **destinatario** può verificare l'autenticità della firma utilizzando la **chiave pubblica** corrispondente (meccanismo inverso rispetto a prima).

Il funzionamento coinvolge diverse fasi descritte di seguito:



# Firma digitale 2/5

## 1. Generazione delle chiavi:

- Il mittente genera una coppia di chiavi: una chiave privata e una chiave pubblica.
- La chiave privata deve essere mantenuta segreta e non deve essere divulgata.
- La chiave pubblica viene diffusa ampiamente e resa disponibile a chiunque voglia verificare le firme digitali del mittente.



# Firma digitale 3/5

## 2. Creazione della firma:

- Il mittente prende il documento che desidera firmare digitalmente.
- Applica una funzione di hash crittografica al documento per generare un valore univoco chiamato "hash" del documento.
- Successivamente, utilizza la sua ***chiave privata per crittografare l'hash del documento***.

**N.B.:** Il risultato della crittografia è la firma digitale del documento.  
Il firmatario usa la chiave privata per firmare.





# Firma digitale 4/5

## 3. Verifica della firma:

- Il destinatario riceve il documento firmato e la firma digitale associata.
- Calcola l'hash del documento utilizzando la stessa funzione di hash crittografica utilizzata dal mittente.
- Utilizza la chiave pubblica del mittente per decifrare la firma digitale.
- Se la decrittografia ha successo e l'hash calcolato corrisponde all'hash decifrato, allora la firma viene considerata valida. In tal caso, il destinatario può confermare che il documento non è stato alterato dopo la firma e che proviene dal mittente specificato.



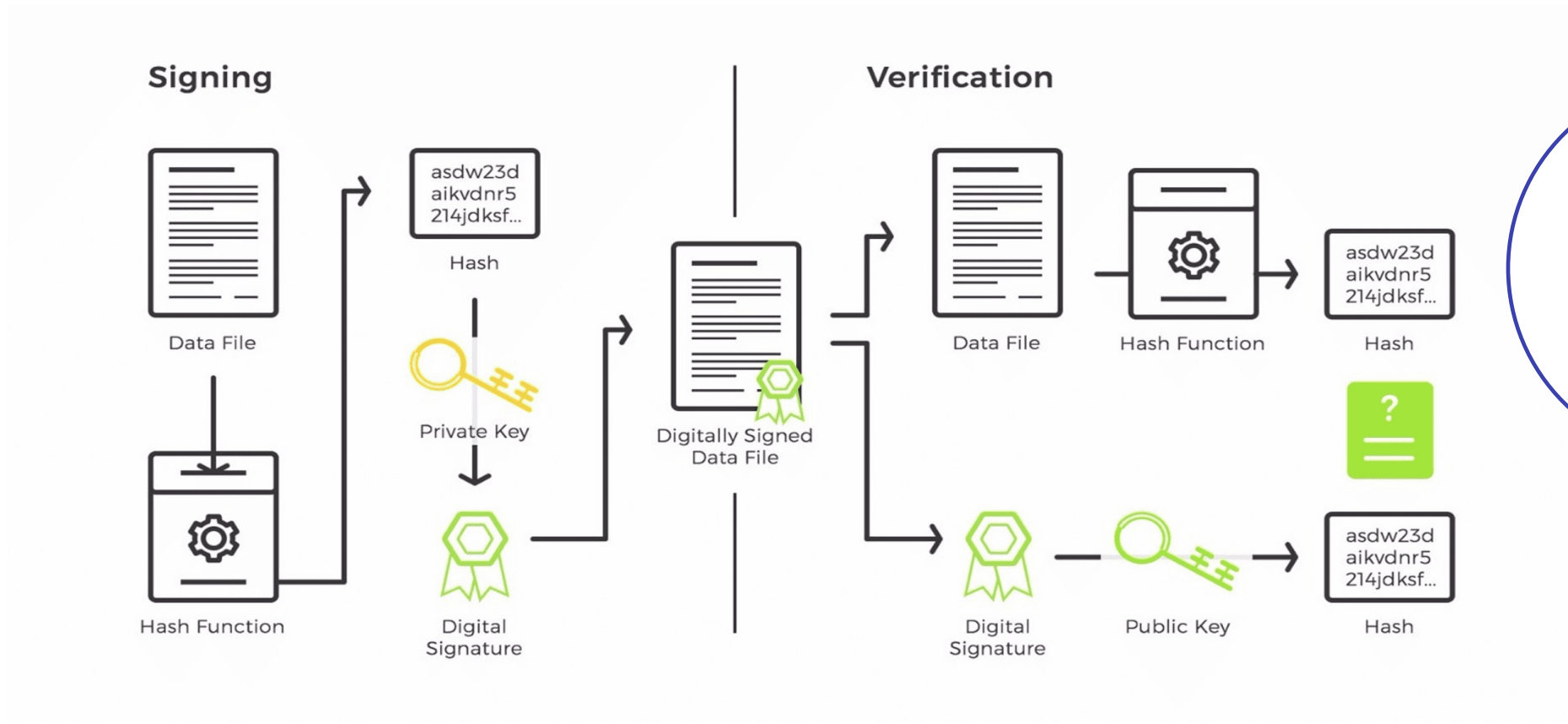
# Firma digitale 5/5

Come accennato, la firma digitale fornisce tre importanti proprietà:

1. **Autenticità:** garantisce l'autenticità del mittente. Il destinatario può verificare che il documento provenga da una **fonte specifica**, poiché solo il mittente conosce la chiave privata necessaria per creare la firma digitale corrispondente.
2. **Integrità:** consente di verificare l'integrità del documento. Qualsiasi modifica apportata al documento dopo la firma verrà rilevata durante la verifica della firma, poiché l'hash del documento modificato non corrisponderà all'hash decifrato dalla firma digitale.
3. **Non ripudiabilità:** impedisce al mittente di negare l'autenticità o l'invio del documento firmato. Una volta che il documento è stato firmato digitalmente, il mittente non può negare la firma in seguito, poiché la verifica della firma può dimostrare la sua partecipazione.



# Firma digitale - schema



# Certificati 1/5

**I certificati a chiave pubblica** (Public Key Certificates) sono documenti digitali che vengono emessi da un'autorità di certificazione (Certification Authority o CA) per associare una chiave pubblica a un'entità specifica, come una persona, un'organizzazione o un dispositivo.

Essi svolgono un ruolo fondamentale nella crittografia a chiave pubblica e nella sicurezza delle comunicazioni.

Sono basati su infrastrutture a chiave pubblica (Public Key Infrastructure o PKI), che costituiscono un insieme di tecnologie, protocolli e procedure che consentono la gestione sicura delle chiavi pubbliche e private.



# Certificati 2/5

I certificati di chiave pubblica contengono le seguenti informazioni:

1. **Identità dell'entità:** Il certificato contiene informazioni sull'identità dell'entità associata alla chiave pubblica, come il nome, l'indirizzo e l'organizzazione.
2. **Chiave pubblica:** Il certificato include la chiave pubblica dell'entità, che viene utilizzata per **crittografare** i messaggi destinati a quella specifica entità.
3. **Firma digitale:** Il certificato è firmato digitalmente dall'autorità di certificazione, conferendo validità e autenticità all'associazione tra la chiave pubblica e l'identità dell'entità. La firma digitale garantisce che il certificato non sia stato alterato e che sia stato emesso da un'autorità di certificazione attendibile.



# Certificati 3/5

L'utilizzo dei certificati di chiave pubblica offre i seguenti vantaggi:

- **Autenticazione:** I certificati di chiave pubblica consentono di verificare l'autenticità dell'entità associata alla chiave pubblica. Gli utenti possono confidare che la chiave pubblica appartiene effettivamente all'entità specificata nel certificato.
- **Crittografia sicura:** Utilizzando la chiave pubblica contenuta nel certificato, gli utenti possono crittografare i messaggi in modo sicuro, garantendo che solo l'entità associata alla chiave privata corrispondente possa decifrarli.
- **Integrità e non ripudiabilità:** La firma digitale presente nel certificato garantisce l'integrità del certificato stesso e la non ripudiabilità dell'associazione tra chiave pubblica e identità. L'entità non può negare la responsabilità di utilizzare la chiave privata associata al certificato.



# Certificati 4/5

I certificati di chiave pubblica sono ampiamente utilizzati in diversi contesti, come la sicurezza delle **comunicazioni web** (HTTPS), la firma digitale, l'autenticazione degli utenti e la crittografia dei dati. Forniscono una struttura di fiducia per la gestione delle chiavi pubbliche e consentono una comunicazione sicura e affidabile tra le parti interessate.

In generale, gli utilizzi dei certificati di chiave pubblica vanno oltre l'autenticazione degli utenti e la crittografia delle comunicazioni. Essi possono essere applicati anche per la firma digitale, la gestione degli accessi, la sicurezza delle transazioni e altre funzionalità che richiedono la validazione dell'identità e la crittografia sicura delle informazioni.

Come si crea un certificato? SimpleAuthority, a mano...



# Certificati 5/5

## Approfondimento:

Nello scambio di informazioni tra client e server, il **client utilizza la chiave pubblica** (contenuta all'interno del certificato) per crittografare i dati che invia al server. Successivamente, **il server** utilizza la corrispondente **chiave privata** per decrittografare i dati ricevuti dal client, dimostrando di essere autorizzato a decifrare i messaggi.

Dopo aver autenticato il server, viene stabilito un canale di comunicazione sicuro tra il server e il client. Questo canale è crittografato utilizzando un algoritmo di crittografia a **chiave simmetrica**, in cui la chiave simmetrica viene generata in modo casuale per ogni sessione.





# Test 2

1. Quale delle seguenti affermazioni descrive correttamente la crittografia?
  - a. Un processo per rendere i dati incomprensibili a chi non è autorizzato ad accedervi.
  - b. Un metodo per accelerare la velocità di trasmissione dei dati.
  - c. Una tecnica per aumentare la capacità di archiviazione dei dati.
  - d. Un algoritmo per aumentare la larghezza di banda di una connessione di rete.
2. Qual è l'obiettivo principale della crittografia?
  - a. Garantire l'integrità dei dati
  - b. Assicurare la disponibilità dei dati.
  - c. Proteggere la riservatezza dei dati.
  - d. Migliorare le prestazioni di elaborazione dei dati.
3. Quale dei seguenti algoritmi di crittografia è considerato sicuro per l'utilizzo generale?
  - a. DES
  - b. RSA
  - c. MD5
  - d. SHA-1



# Test 2

4. Cos'è un attacco di forza bruta?
  - a. Un attacco in cui un attaccante cerca di indovinare una password.
  - b. Un attacco in cui un attaccante sfrutta una debolezza nel sistema operativo.
  - c. Un attacco in cui un attaccante invia una grande quantità di dati alla rete.
  - d. Un attacco in cui un attaccante altera il flusso di dati durante la comunicazione.
5. Qual è il ruolo di una chiave crittografica nella crittografia simmetrica?
  - a. Decifrare i dati cifrati.
  - b. Firmare digitalmente i dati.
  - c. Crittografare i dati.
  - d. Verificare l'autenticità dei dati.



# Test 2

## Soluzioni:

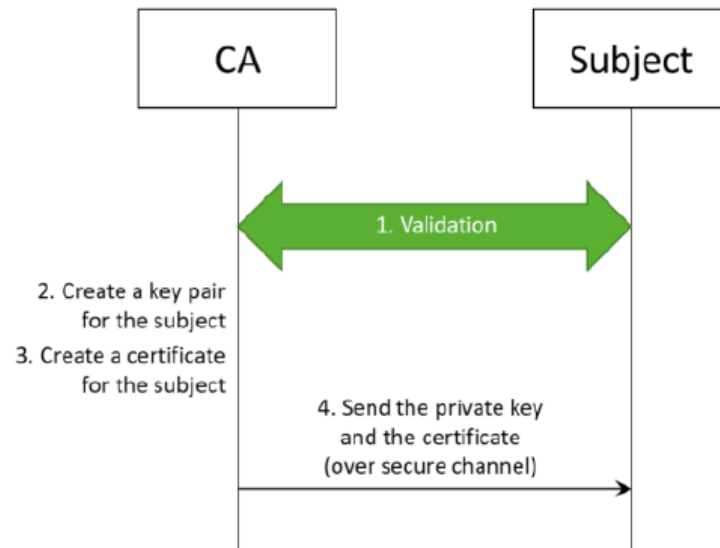
1. a) Un processo per rendere i dati incomprensibili a chi non è autorizzato ad accedervi.
2. c) Proteggere la riservatezza dei dati.
3. b) RSA
4. a) Un attacco in cui un attaccante cerca di indovinare una password.
5. a) + c) Decifrare e Crittografare i dati.



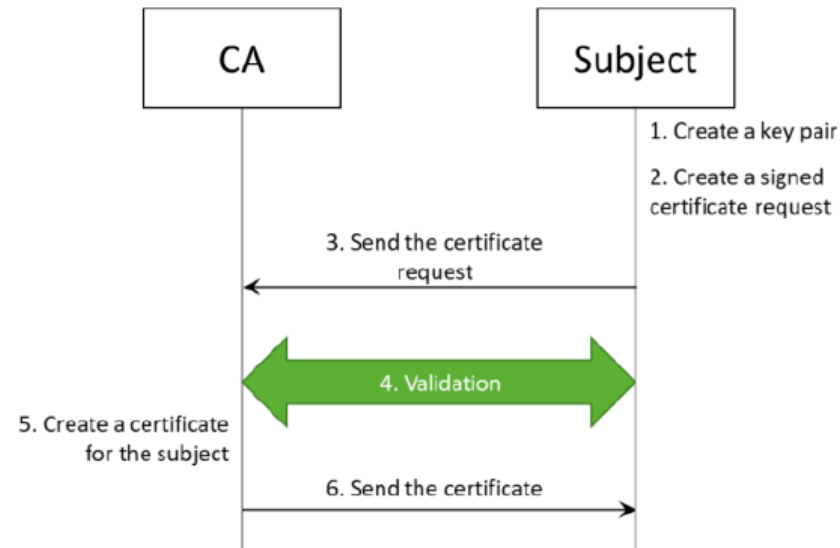
# Approfondimento

## Creating Certificates

### • Simple way:



### • Secure way:



# Approfondimento

## Linux - Mac

Per creare un certificato, una chiave privata e un'autorità di certificazione (CA) su Linux, puoi seguire i passaggi seguenti:

- Installare OpenSSL (se non presente: > openssl)
  - yum -y install openssl
  - apt-get install openssl
- Generare la chiave privata della CA
  - openssl genrsa -out ca.key 2048
- Generare il certificato della CA
  - openssl req -new -x509 -days 365 -key ca.key -out CA\_cert.pem
- Generare la chiave per il server
  - openssl genrsa -out signer\_prvkey.pem 2048



# Approfondimento

- Generare CSR per il server (da dare alla CA)
  - `openssl req -new -key signer_prvkey.pem -out server.csr`
- Firmare il CSR e generare il certificato finale del server
  - `openssl x509 -req -days 365 -in server.csr -CA CA_cert.pem -CAkey ca.key -CAcreateserial -out signer_cert.pem`

*Solitamente all'out di ogni comando le estensioni non sono tutte ".pem", si utilizzano diverse estensioni: ".key", ".crt", ".csr".*

*Durante il processo di creazione del CSR, verranno richieste alcune informazioni, come il nome comune (CN), l'organizzazione, ecc. Compila attentamente questi campi*



# Sicurezza Informatica – U2

## II lezione



# Sicurezza Informatica – U2

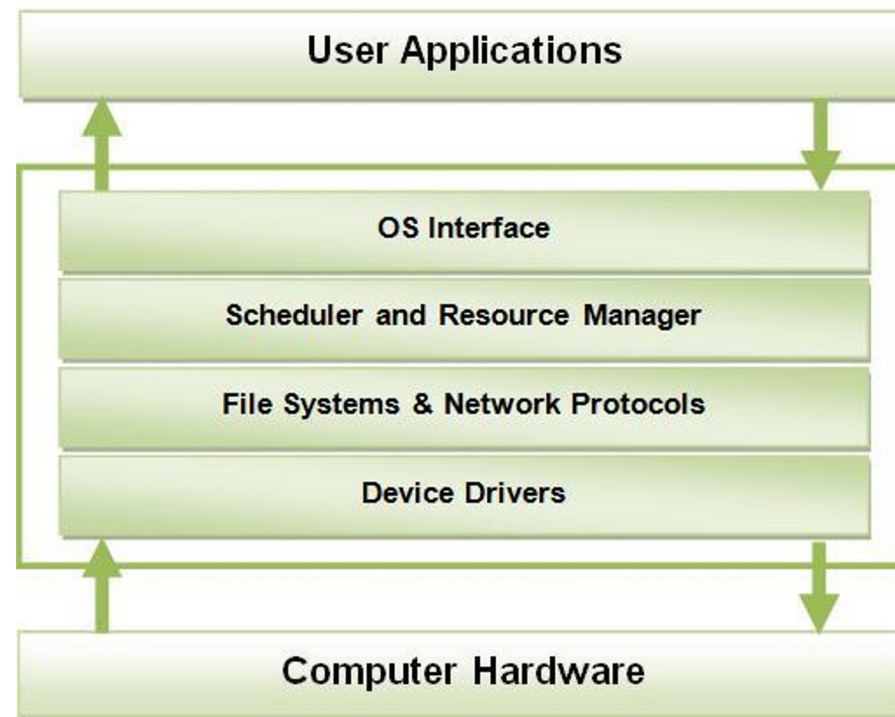
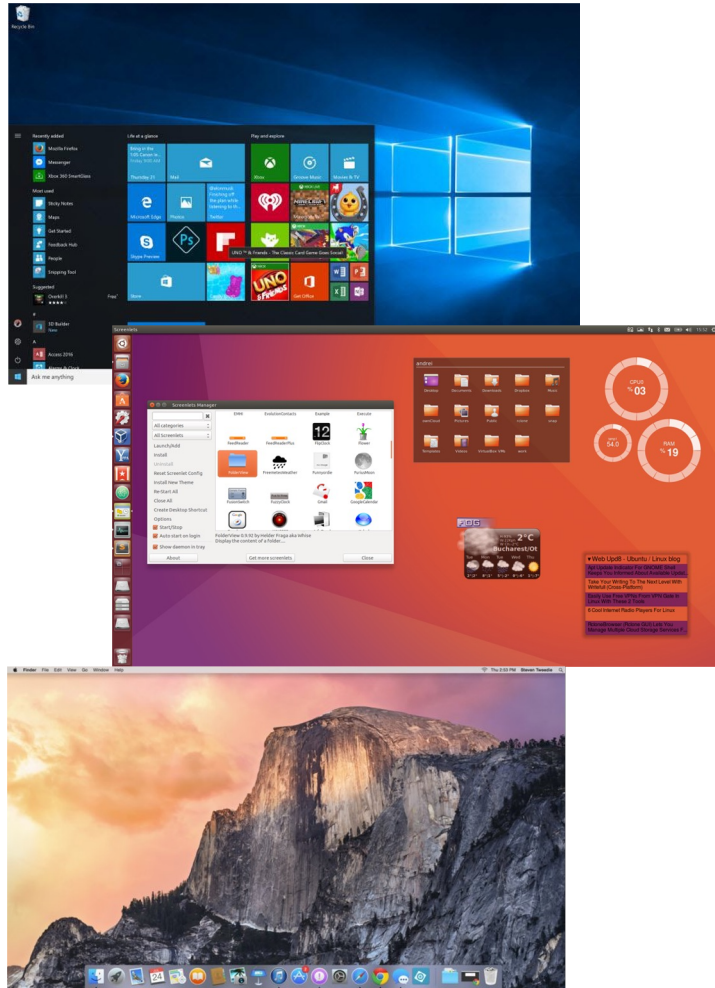
## Sommario II lezione

- Il Sistema Operativo
- Access Control Lists
- Modelli di sicurezza Unix/Linux
- La Sicurezza del Sistema Operativo
- Malware e contromisure

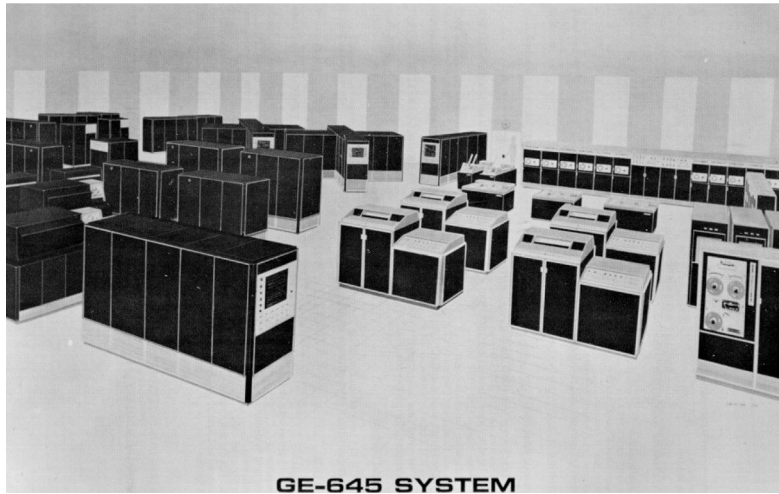




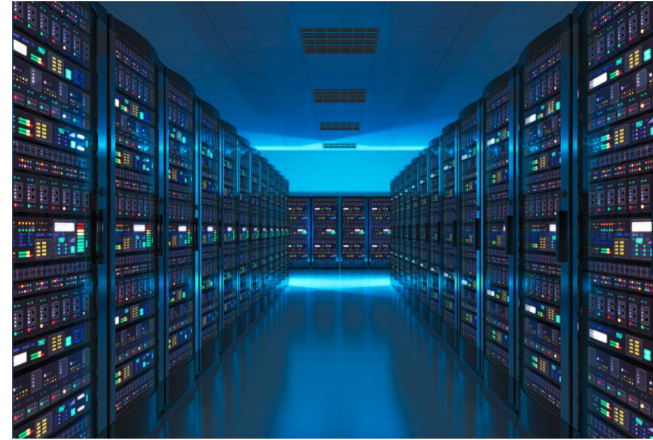
# Sistemi Operativi 1/2



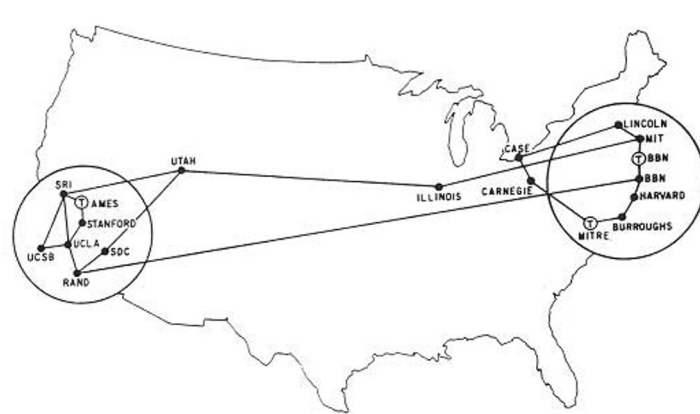
# Sistemi Operativi 2/2



Timesharing



Cloud



Reti



# Modellizzazione

## ***Soggetti che compiono azioni su oggetti***

**Soggetti:** utenti, processi, programmi, ...

**Oggetti:** file, database, ...

**Azioni:** leggere, scrivere, cambiare permessi, ...

## **Specifica di sicurezza:**

### **Security Goals**

*Segretezza:* chi può leggere cosa

*Integrità:* chi può modificare cosa

*Disponibilità:* garanzie sull'accesso alle risorse

**Trusted Computing Base:** Hw/Sw di cui è necessario fidarsi

**Threat model:** Modello di rischio, cosa può fare l'attaccante?



# Algoritmo ACL

Quando un processo cerca di accedere un file, la ACL viene applicata nel seguente ordine:

1. Se l'utente del processo corrisponde ad una delle entrate user, l'entrata in questione viene utilizzata per verificare i permessi necessari
  - a) Se l'utente non e' proprietario, viene applicata l'eventuale Maschera
2. Se esiste almeno una entrata group corrispondente ad uno dei gruppi del processo, vengono considerate tutte queste entrate per verificare se almeno una di esse fornisce il permesso necessario (una volta applicata la maschera)
3. Altrimenti, l'entrata other viene utilizzata per verificare i permessi necessari



# ACL – Access Control Lists

Il protection state è memorizzato per colonne:

- file 1: processo 1 (read); processo 2 (read)
- file 2: processo 2 (read/write)
- file 3: processo 1 (read/write); processo 2 (read)



**Protection  
state**

|            | file 1 | file 2     | file 3     |
|------------|--------|------------|------------|
| processo 1 | read   | -          | read/write |
| processo 2 | read   | read/write | read       |

# Capabilities

Il protection state è memorizzato per righe:

processo 1: read file1; read/write file 3

processo 2: read file 1; read/write file 2; read file 3

**Protection  
state**

|            | file 1 | file 2     | file 3     |
|------------|--------|------------|------------|
| processo 1 | read   | -          | read/write |
| processo 2 | read   | read/write | read       |

# Discretionary vs Mandatory Protection Systems

I sistemi di protezione discrezionale e i sistemi di protezione obbligatoria sono due approcci diversi per l'applicazione dei controlli di accesso e delle politiche di sicurezza nei sistemi informatici. Ecco una breve spiegazione di ciascuno:

1. **Sistemi di protezione discrezionale:** *DAC* è un modello di sicurezza in cui il proprietario o l'utente del sistema ha la discrezione o il controllo sulle autorizzazioni di accesso alle risorse. Il sistema consente al proprietario delle risorse di determinare chi può accedere alle sue risorse e quale livello di accesso hanno. Le decisioni di controllo di accesso si basano sulla discrezione del proprietario delle risorse o di un amministratore che può modificare le autorizzazioni di accesso. Gli utenti possono concedere o revocare i privilegi di accesso alle proprie risorse a loro discrezione. Esempi di sistemi di protezione discrezionale includono le autorizzazioni a livello di file nei sistemi operativi come Unix o Windows.





# Discretionary vs Mandatory Protection Systems

2. **Sistemi di protezione obbligatoria:** I sistemi di protezione obbligatoria (Mandatory Access Control, *MAC*) sono basati su politiche di sicurezza più rigide e vincolanti rispetto ai sistemi di protezione discrezionale. In questi sistemi, l'accesso alle risorse è determinato da regole e politiche di sicurezza predefinite, che sono imposte in modo coerente su tutti gli utenti e le risorse del sistema.
- Le decisioni di accesso sono basate su criteri come il grado di sicurezza, l'etichetta di sicurezza o altre classificazioni dei dati. I sistemi di protezione obbligatoria sono spesso utilizzati in ambienti ad alto rischio o sensibili, come governi o organizzazioni militari, per garantire un controllo più rigoroso e granulare sull'accesso alle risorse.

In sintesi, i sistemi di protezione discrezionale concedono ai proprietari delle risorse il *controllo discrezionale sull'accesso*, mentre i sistemi di protezione obbligatoria si basano su *politiche di sicurezza predefinite*.





# La Sicurezza in pratica

## Scomporre i diritti di root

- *Dropping privileges*
- *Linux capabilities*

## Confinare i danni (*sandboxing*)

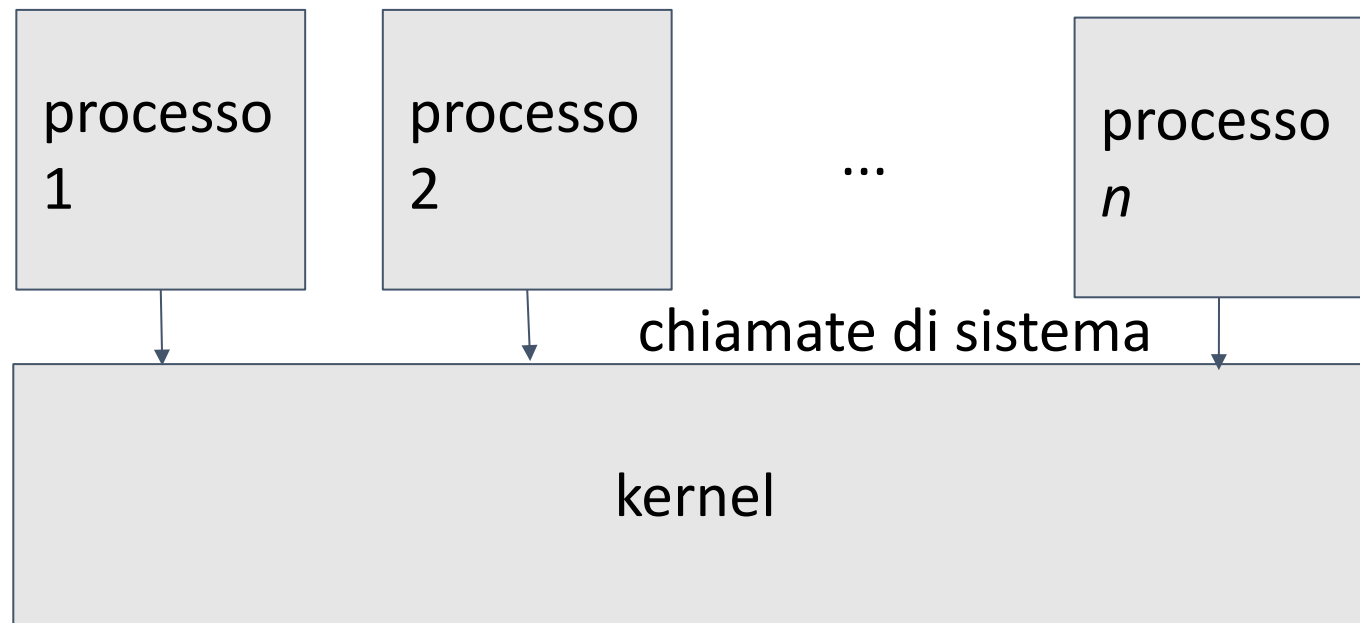
- *Hardware: Virtual Machines/ARM TrustZone*
- *Software: chroot/Containers/Zones/Jails*

## Retrofitting MAC in un kernel tradizionale

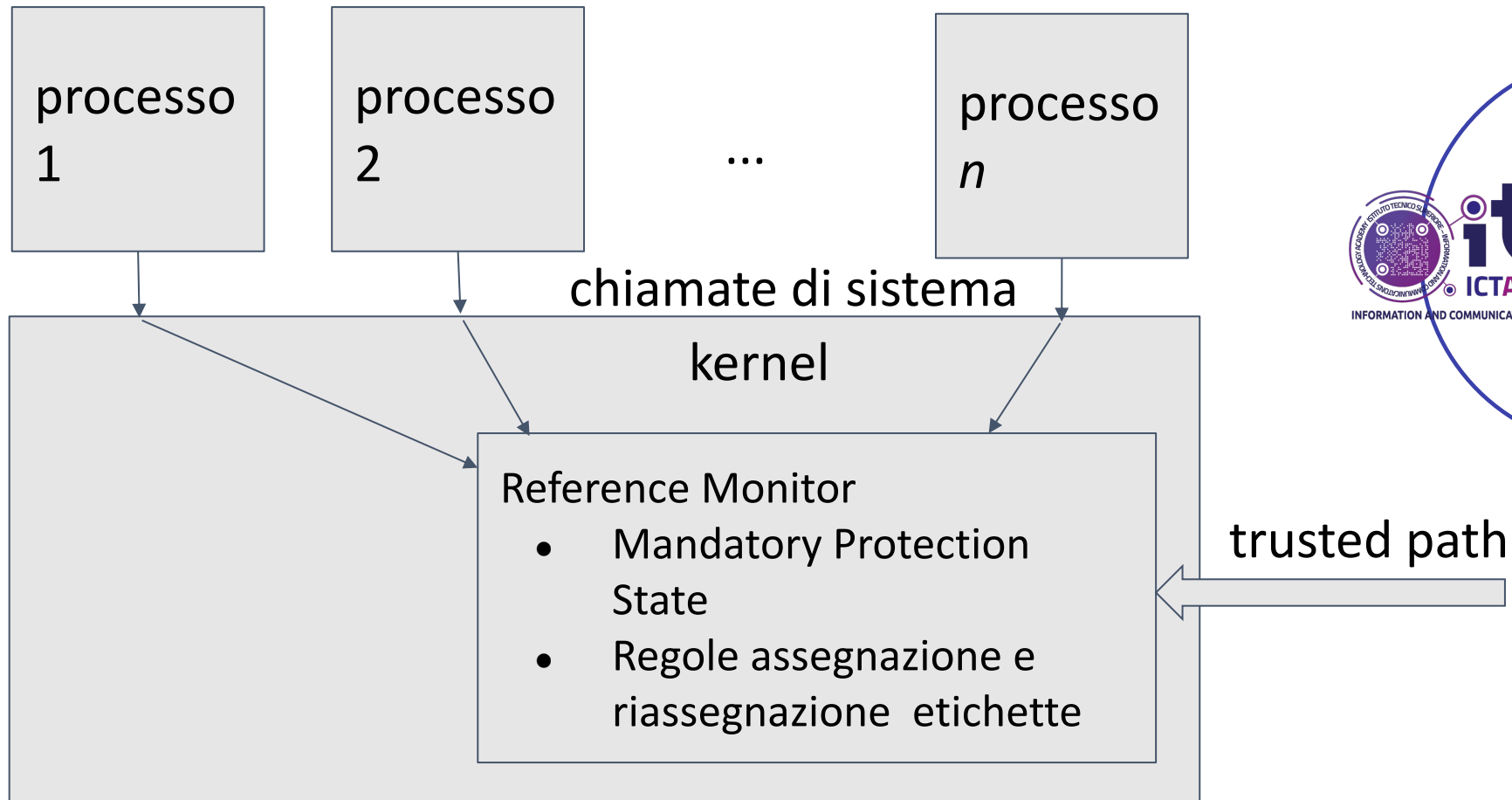
- *AppArmor, SELinux*



# Kernel classico



# Kernel Classico + Reference Monitor



# Malware e contromisure 1/7

Il malware è un termine generico che descrive **software dannoso** progettato per infettare, danneggiare o ottenere un accesso non autorizzato a un sistema informatico.

Esistono diverse categorie di malware, tra cui virus, worm, trojan, spyware, adware e ransomware. Ogni tipo di malware ha obiettivi e comportamenti specifici, ma tutti rappresentano una minaccia per la sicurezza dei sistemi e dei dati.. Di seguito sono riportati alcuni esempi comuni di malware:

- **Virus:** Un virus informatico è un tipo di malware che si collega a un *file* esistente o a un programma eseguibile e si diffonde infettando altri file o programmi. Può causare danni al sistema, eliminare o corrompere dati e replicarsi in modo autonomo.



# Malware e contromisure 2/7

- **Worm:** Un worm è un tipo di malware autonomo che si diffonde in modo automatico attraverso le reti informatiche, sfruttando le vulnerabilità dei sistemi. Può replicarsi *senza bisogno di un file ospite* e può causare gravi danni, tra cui rallentamento della rete e saturazione delle risorse.
- **Trojan:** Un trojan o cavallo di Troia è un malware che si presenta come un *software legittimo* o inoffensivo, ma una volta eseguito, svolge attività dannose senza il consenso dell'utente. Può consentire l'accesso remoto al sistema, rubare informazioni sensibili o *aprire una porta di ingresso* per altri malware.



# Malware e contromisure 3/7

- **Spyware:** Il spyware è un tipo di malware progettato per *raccogliere informazioni* senza il consenso dell'utente. Può monitorare l'attività dell'utente, raccogliere dati personali come password, informazioni finanziarie o cronologia di navigazione e inviarli a terze parti.
- **Adware:** L'adware è un tipo di malware che mostra annunci pubblicitari indesiderati sul sistema infetto. Può rallentare il sistema, interrompere la normale esperienza di navigazione e persino raccogliere informazioni sulle preferenze dell'utente per scopi di marketing.
- **Ransomware:** Il ransomware è un tipo di malware che *blocca o crittografa* i dati del sistema e richiede un riscatto per ripristinare l'accesso ai file. Può causare gravi danni finanziari e interrompere le attività aziendali o personali.



# Malware e contromisure 4/7

- **Rootkit:** Un rootkit è un tipo di malware che ottiene un accesso privilegiato al sistema, generalmente ottenendo privilegi di amministratore o "root". Può nascondere la sua presenza e consentire a un attaccante di accedere e controllare il sistema in modo non autorizzato.

Questi sono solo alcuni esempi di malware, ma l'elenco è in continuo sviluppo poiché gli attaccanti creano costantemente nuove varianti e tipologie di malware per eludere le misure di sicurezza.

È importante adottare precauzioni e utilizzare soluzioni di sicurezza aggiornate per proteggere i sistemi e i dati dalle minacce malware.



# Malware e contromisure 5/7

Per contrastare il malware e proteggere i sistemi informatici, è importante adottare una serie di contromisure, tra cui:

- **Software antivirus:** Installare e mantenere un software antivirus aggiornato è fondamentale per rilevare e rimuovere il malware. Gli antivirus utilizzano algoritmi e database per identificare e bloccare le minacce conosciute.
- **Aggiornamenti del sistema operativo e del software:** Mantenere il sistema operativo e il software applicativo aggiornati con gli ultimi patch di sicurezza è essenziale per rimediare alle vulnerabilità note che possono essere sfruttate dai malware.





# Malware e contromisure 6/7

- **Firewall:** Utilizzare un firewall per controllare il traffico di rete in ingresso e in uscita può aiutare a bloccare l'accesso non autorizzato ai sistemi e a impedire la propagazione del malware.
- **Politiche di sicurezza** e sensibilizzazione degli utenti: Implementare politiche di sicurezza solide che includano password complesse, autenticazione a due fattori e limitazioni di accesso. Inoltre, educare gli utenti sull'importanza di evitare clic su link o allegati sospetti e di prestare attenzione alle pratiche di sicurezza online.
- **Backup regolari:** Eseguire regolarmente copie di backup dei dati critici e mantenerle in una posizione sicura. In caso di infezione da malware o ransomware, sarà possibile ripristinare i dati da un backup pulito.



# Malware e contromisure 7/7

- **Monitoraggio delle attività:** Implementare strumenti di monitoraggio delle attività di rete e dei sistemi per rilevare e rispondere tempestivamente alle attività sospette o anomale.
- **Protezione delle e-mail:** Utilizzare filtri antispam e soluzioni di sicurezza per e-mail per bloccare i messaggi di phishing e impedire l'accesso ai link o agli allegati malevoli.

Queste contromisure lavorano sinergicamente per mitigare i rischi associati al malware e proteggere i sistemi informatici.

È importante mantenere una pratica costante di sicurezza e rimanere aggiornati sulle ultime minacce informatiche per adattare le contromisure di conseguenza.



# Test 3

Qual è l'obiettivo principale di una politica di controllo degli accessi?

- a) Garantire la disponibilità dei dati
- b) Proteggere la riservatezza dei dati
- c) Garantire l'integrità dei dati
- d) Tutte le precedenti

Quali sono i tre principali tipi di controlli di accesso?

- a) Identificazione, autenticazione, autorizzazione
- b) Crittografia, firma digitale, firewall
- c) Antivirus, firewall, backup
- d) Controllo di versione, registrazione delle attività, sicurezza fisica



# Test 3

Soluzioni:

1. d) Tutte le precedenti
2. a) Identificazione, autenticazione, autorizzazione



# Approfondimenti

Quali sono i migliori strumenti per verificare file, email etc:

- a) Virustotal: piattaforma multi-engine per file, URL, ip
- b) MetaDefender: piattaforma multi-engine per file, URL e ip
- c) Hybrid Analysis: effettua sia l'analisi dinamica eseguendo i file in un ambiente sandbox controllato, che l'analisi statica e di rete.
- d) Jotti's Malware Scan: consente di caricare file sospetti per una scansione multi-engine.
- e) Any.Run: piattaforma di analisi che consente di eseguire i file in un ambiente sandbox controllato e osservarne il comportamento in tempo reale.
- f) Cuckoo Sandbox: framework di analisi dinamica che permette di eseguire i file in un ambiente sandbox controllato per monitorare le attività dannose per file o URL sospetti.



# Approfondimenti

1. Rootme – root-me.org
2. WAPT Challenges in generale
3. Write a client program and a server program (logically only). The server can send authenticated and secret messages to the client. The server has a private key and a certificate.

The server:

1. Takes from keyboard a message to encrypt
2. Encrypts the message with AES-128-CBC with a key hardcoded (32 characters) in the script (= ciphertext).
3. Loads the private key (read “rb”).
4. Signs the ciphertext (and the IV) with such a private key
5. Saves the signature, the IV (Initialized Vector), and the ciphertext on a “message file”, (message.enc)



# Approfondimenti

The client:

1. Loads the server's and the CA's certificates (read "rb")
2. Verifies the validity of the server certificate (only signature)
3. Loads the signature, the IV, and the ciphertext from the "message file" (message.enc)
4. Verifies the signature with the public key embedded in the certificate
5. Decrypts the message
6. Prints the message on screen



# Sicurezza Informatica – U2

## III lezione





# Sicurezza Informatica – U2

## Sommario III lezione

- Sicurezza dei sistemi informatici:
  - Attacchi (D)DOS e contromisure
  - Riconoscimento di intrusioni
  - Prevenzione di intrusioni (firewall)
- Sicurezza nel software (codice):
  - Stack overflow
  - Gestione dell'input utente
  - Interazioni tra programmi
  - Gestione dell'output



# Attacchi (D)DoS e contromisure

Gli attacchi di tipo Denial of Service (DoS) mirano a sovraccaricare o saturare le risorse di un sistema, rendendolo inaccessibile agli utenti legittimi. Di seguito sono riportati alcuni esempi di attacchi DoS e alcune contromisure che possono essere adottate per mitigarli:

**Attacco Flood:** In questo tipo di attacco, vengono inviati un gran numero di ***richieste legittime*** al sistema target, sovraccaricandolo e impedendo agli utenti legittimi di accedere alle risorse. Le contromisure possono includere l'implementazione di sistemi di rilevamento e prevenzione delle intrusioni (IDS/IPS) per identificare e bloccare il traffico anomalo, nonché l'implementazione di limiti di velocità e limiti di connessione per mitigare l'impatto dell'attacco.



# Attacchi (D)DoS e contromisure

**Attacco DDoS** (Distributed Denial of Service): In un attacco DDoS, un grande numero di dispositivi (spesso compromessi) invia contemporaneamente richieste al sistema target, sovraccaricandolo. Le contromisure possono includere l'utilizzo di servizi di mitigazione DDoS che filtrano il traffico dannoso, l'implementazione di sistemi di bilanciamento del carico per distribuire le richieste in modo equo e l'uso di firewall e router configurati correttamente per bloccare il traffico sospetto.



# Attacchi (D)DoS e contromisure

**Attacco di amplificazione:** In questo tipo di attacco, l'attaccante sfrutta server o dispositivi vulnerabili per inviare una **grande quantità** di dati al sistema target, amplificando l'effetto dell'attacco. Le contromisure possono includere l'aggiornamento e la corretta configurazione dei server e dei dispositivi di rete per evitare l'amplificazione, nonché la filtrazione del traffico sospetto a livello di rete.



# Attacchi (D)DoS e contromisure

**Attacco di esaurimento delle risorse:** Questo tipo di attacco mira a esaurire le risorse del sistema target, come la CPU, la memoria o la larghezza di banda di rete. Le contromisure possono includere l'implementazione di limiti di utilizzo delle risorse, il monitoraggio delle prestazioni del sistema per rilevare anomalie e l'uso di algoritmi di gestione delle risorse efficaci.



# Attacchi (D)DoS e contromisure

**Attacco di vulnerabilità di protocollo:** Questo tipo di attacco sfrutta le vulnerabilità dei protocolli di rete per interrompere o compromettere la comunicazione. Le contromisure possono includere l'applicazione regolare di patch e aggiornamenti per correggere le vulnerabilità, nonché la configurazione corretta dei dispositivi di rete per mitigare gli effetti dell'attacco.



# Attacchi (D)DoS e contromisure

È importante notare che la protezione contro gli attacchi DoS richiede una combinazione di contromisure tecniche, come la configurazione dei dispositivi di rete e l'implementazione di sistemi di difesa, e una pianificazione adeguata della capacità per garantire che il sistema possa gestire carichi di traffico elevati.

Infatti, per mitigare gli attacchi DoS (Denial of Service), esistono diverse contromisure che possono essere adottate.

Di seguito sono elencate alcune delle principali contromisure:

- 1. Filtraggio del traffico:** Utilizzare firewall, router o dispositivi di mitigazione DDoS per filtrare il traffico dannoso proveniente dagli attaccanti. Questa misura aiuta a bloccare le richieste dannose e a consentire solo il traffico legittimo verso il sistema target.



# Attacchi (D)DoS e contromisure

- 2. Configurazione dei limiti di velocità:** Impostare limiti di velocità per le connessioni in entrata per evitare che un singolo indirizzo IP o un singolo utente saturi la larghezza di banda. Questo aiuta a bilanciare il carico e a prevenire attacchi di saturazione della rete.
- 3. Controllo del traffico:** Implementare sistemi di rilevamento delle intrusioni (IDS) o sistemi di prevenzione delle intrusioni (IPS) per monitorare il traffico in tempo reale e identificare comportamenti sospetti o anomali. Questi sistemi possono aiutare a rilevare e bloccare gli attacchi DoS in corso.





# Attacchi (D)DoS e contromisure

4. **Load balancing:** Utilizzare sistemi di bilanciamento del carico per distribuire le richieste di traffico tra più server o risorse. Questo aiuta a distribuire il carico di lavoro e a prevenire che un singolo server o risorsa venga sovraccaricato.
5. **DDoS mitigation service:** Considerare l'utilizzo di servizi di mitigazione DDoS da fornitori specializzati. Questi servizi offrono una protezione avanzata contro gli attacchi DoS e DDoS, filtrando il traffico dannoso prima che raggiunga il sistema target.
6. **Monitoraggio delle prestazioni:** Monitorare costantemente le prestazioni del sistema per rilevare eventuali anomalie o sovraccarichi. Questo consente di identificare rapidamente gli attacchi in corso e di adottare misure correttive tempestive.



# Attacchi (D)DoS e contromisure

7. **Pianificazione della capacità:** Assicurarsi che il sistema abbia la capacità di gestire carichi di traffico elevati. Effettuare una pianificazione adeguata della capacità, ad esempio attraverso l'aggiunta di risorse o l'uso di servizi di cloud computing scalabili, può aiutare a prevenire gli effetti dannosi degli attacchi DoS.
8. **Protezione dei protocolli di rete:** Configurare correttamente i dispositivi di rete e gli applicativi per prevenire attacchi che sfruttano vulnerabilità dei protocolli di rete. Applicare regolarmente patch e aggiornamenti per correggere le vulnerabilità note.



# Attacchi (D)DoS e contromisure

9. **Backup e ripristino dei dati:** Effettuare regolarmente il backup dei dati critici e avere procedure di ripristino dei dati in caso di interruzioni o danni causati da attacchi DoS. Ciò aiuta a garantire la continuità delle operazioni anche in seguito a un attacco.
10. **Consapevolezza e formazione:** Sensibilizzare gli utenti e il personale sulle minacce degli attacchi DoS e fornire formazione sulla sicurezza informatica per garantire che siano consapevoli delle buone pratiche e dei comportamenti sicuri da adottare.

Come anticipato, tutte queste contromisure possono essere adottate in combinazione per proteggere efficacemente un sistema dai pericolosi attacchi DoS e mitigarne gli effetti negativi sulla disponibilità delle risorse e dei servizi.



# Test 4

Qual è il significato dell'acronimo "DDoS"?

- a) Distributed Denial of Service
- b) Data Disclosure of Sensitive information
- c) Direct Detection of System vulnerabilities
- d) Digital Data on Secure servers

Quali delle seguenti tecniche è utilizzata per proteggere una rete da attacchi di "spoofing"?

- a) Firewall
- b) Antivirus
- c) Crittografia
- d) Sistema di rilevamento delle intrusioni (IDS)



# Test 4

Cosa indica il termine "phishing" nel contesto della sicurezza informatica?

- a) Un attacco in cui un aggressore ottiene accesso non autorizzato a un sistema utilizzando una combinazione di nome utente e password.
- b) Un attacco in cui un aggressore invia e-mail o messaggi ingannevoli per ottenere informazioni personali o finanziarie.
- c) Un attacco in cui un aggressore manipola il traffico di rete per ottenere accesso a dati sensibili.
- d) Un attacco in cui un aggressore intercetta e decifra la comunicazione tra un client e un server.



# Test 4

Qual è lo scopo principale di un sistema di rilevamento delle intrusioni (IDS)?

- a) Monitorare il traffico di rete e rilevare attività sospette o non autorizzate.
- b) Crittografare i dati sensibili per proteggerli durante il trasferimento.
- c) Bloccare l'accesso a siti web dannosi o non sicuri.
- d) Proteggere un sistema o una rete da attacchi di tipo "buffer overflow".



# Riconoscimento delle intrusioni

Il riconoscimento delle intrusioni, noto anche come rilevamento delle intrusioni o **Intrusion Detection**, è una componente fondamentale della sicurezza informatica.

Si riferisce alla capacità di identificare e rispondere agli accessi non autorizzati o alle attività sospette all'interno di un sistema o di una rete.

Il riconoscimento delle intrusioni è un elemento chiave per la sicurezza informatica, in quanto consente di identificare tempestivamente attività dannose o non autorizzate e di adottare misure correttive per proteggere il sistema e le informazioni sensibili.



# Riconoscimento delle intrusioni

Il riconoscimento delle intrusioni può essere effettuato attraverso due principali approcci:

1. **Riconoscimento delle intrusioni basato su firme:** Questo approccio si basa sull'utilizzo di firme o modelli predefiniti per identificare specifici schemi o comportamenti noti di attacchi o intrusioni. Si tratta di confrontare il traffico di rete o il comportamento del sistema con una vasta libreria di firme di attacco conosciute. Se una corrispondenza viene trovata, viene generato un avviso o un allarme per segnalare l'attività sospetta.





# Riconoscimento delle intrusioni

2. Riconoscimento delle intrusioni **basato su comportamento**:  
Questo approccio si concentra sul monitoraggio e l'analisi del normale comportamento del sistema o della rete per **identificare deviazioni** o anomalie che potrebbero indicare un'attività non autorizzata. Utilizzando algoritmi avanzati, il sistema è in grado di apprendere il normale comportamento del sistema e rilevare anomalie significative. Le anomalie possono includere pattern di traffico insoliti, utilizzi anomali delle risorse o comportamenti non conformi alle politiche di sicurezza stabilite.



# Riconoscimento delle intrusioni

Alcuni dei metodi e delle tecnologie comunemente utilizzate per il riconoscimento delle intrusioni includono:

- **Sistemi di rilevamento delle intrusioni (IDS):** Questi sistemi monitorano il traffico di rete in tempo reale e rilevano attività sospette o anomale. Possono essere basati su firme o comportamenti.
- **Sistemi di prevenzione delle intrusioni (IPS):** A differenza degli IDS, gli IPS non solo rilevano, ma anche rispondono attivamente agli attacchi o alle intrusioni. Possono bloccare o filtrare il traffico dannoso in tempo reale.



# Riconoscimento delle intrusioni

- **Analisi dei log:** L'analisi dei log del sistema e degli eventi può rivelare attività anomale o tentativi di accesso non autorizzati. L'analisi dei log può essere svolta manualmente o con l'ausilio di strumenti automatizzati.
- **Monitoraggio dei comportamenti degli utenti:** Il monitoraggio del comportamento degli utenti all'interno di un sistema o di una rete può rilevare azioni sospette o comportamenti insoliti che potrebbero indicare un'attività non autorizzata.



# Prevenzione delle intrusioni

La prevenzione delle intrusioni (Intrusion Prevention) è una strategia di sicurezza informatica che mira a **impedire gli accessi** non autorizzati e gli attacchi informatici al sistema o alla rete. L'obiettivo principale è quello di rilevare, bloccare e mitigare le intrusioni in tempo reale, prima che possano causare danni significativi.

La prevenzione delle intrusioni è un aspetto cruciale della sicurezza informatica per garantire la protezione dei sistemi e delle reti da attacchi dannosi. Implementando una combinazione di misure tecniche e pratiche di sicurezza, è possibile ridurre significativamente il rischio di violazioni e danni alle informazioni sensibili.



# Prevenzione delle intrusioni

Le misure di prevenzione delle intrusioni includono una combinazione di tecniche e soluzioni tecnologiche, tra cui:

**Sistemi di prevenzione delle intrusioni (IPS):** Questi sistemi monitorano il traffico di rete in tempo reale per individuare e prevenire attività sospette o non autorizzate. Possono bloccare o filtrare il traffico dannoso, ad esempio utilizzando firme di attacco o analisi comportamentale.

**Firewall:** I firewall sono dispositivi di sicurezza che monitorano e controllano il traffico di rete tra reti diverse. Possono essere configurati per bloccare il traffico non autorizzato o sospetto in base a regole predefinite.



# Prevenzione delle intrusioni

- **Controllo degli accessi:** Implementare politiche di controllo degli accessi che limitano l'accesso alle risorse del sistema solo a utenti autorizzati. Ciò può includere l'uso di autenticazione forte, come l'autenticazione a due fattori, e la gestione dei privilegi degli utenti.
- **Monitoraggio e analisi dei log:** Il monitoraggio dei log di sistema e di rete consente di rilevare attività sospette o anomalie che potrebbero indicare un tentativo di intrusione. L'analisi dei log può essere automatizzata utilizzando strumenti specializzati.



# Prevenzione delle intrusioni

- **Patch e aggiornamenti:** Mantenere il sistema e il software aggiornati con le ultime patch di sicurezza. Le vulnerabilità note possono essere sfruttate dagli attaccanti, quindi è importante applicare regolarmente gli aggiornamenti per mitigare tali rischi.
- **Educazione e formazione degli utenti:** Sensibilizzare gli utenti sulle best practice di sicurezza informatica e fornire formazione per riconoscere e gestire le potenziali minacce. Gli utenti consapevoli possono contribuire a prevenire le intrusioni attraverso comportamenti sicuri e attenzione ai potenziali rischi.



# Sicurezza nel software (codice)

La sicurezza nel software è un aspetto fondamentale per garantire la protezione dei dati e la stabilità dei sistemi informatici. Consiste nell'adottare pratiche e misure di progettazione, sviluppo e gestione del software per mitigare le vulnerabilità e prevenire potenziali minacce.

Ecco alcuni principali aspetti della sicurezza nel software:

- **Validazione dell'input:** È importante validare l'input dei dati fornito dagli utenti o da altre fonti esterne per prevenire l'inserimento di dati dannosi o non validi che potrebbero essere sfruttati per attacchi come l'iniezione di codice.





# Sicurezza nel software (codice)

- **Gestione degli errori:** Un software sicuro dovrebbe gestire correttamente gli errori e le eccezioni per evitare che possano essere sfruttati dagli attaccanti per ottenere informazioni sensibili o causare danni al sistema.
- **Autenticazione e autorizzazione:** Il software deve implementare meccanismi di autenticazione robusti per verificare l'identità degli utenti e controllare l'accesso alle risorse. Questo include l'utilizzo di password sicure, autenticazione a due fattori e gestione dei privilegi.
- **Crittografia:** La crittografia dei dati sensibili è essenziale per proteggere le informazioni durante il trasferimento e l'archiviazione. Ciò può includere l'utilizzo di algoritmi di crittografia affidabili per proteggere i dati sensibili.



# Sicurezza nel software (codice)

- **Gestione delle sessioni:** Il software dovrebbe gestire correttamente le sessioni degli utenti, compresa la gestione dei token di autenticazione e la protezione contro attacchi come il furto di sessione.
- **Aggiornamenti del software:** Mantenere il software aggiornato con le ultime patch di sicurezza è fondamentale per mitigare le vulnerabilità note. Gli aggiornamenti regolari aiutano a correggere errori e falle di sicurezza e ad affrontare le nuove minacce.
- **Test di sicurezza:** Effettuare test di sicurezza approfonditi, come i test di penetration testing, per identificare e risolvere le vulnerabilità nel software prima che possano essere sfruttate da attacchi esterni.



# Sicurezza nel software (codice)

- **Stack overflow:** lo "stack overflow" è un problema comune nella sicurezza del software che può essere sfruttato dagli attaccanti per eseguire attacchi di tipo buffer overflow. Si verifica quando il limite di memoria riservato per lo stack di un programma viene superato a causa di una ricorsione eccessiva o di una scrittura non controllata nella memoria di stack.
- **Interazioni tra programmi:** È fondamentale garantire che l'interazione tra programmi avvenga in modo sicuro e che i dati trasmessi siano protetti da eventuali minacce o manipolazioni indesiderate. È necessario prestare attenzione ai seguenti aspetti: autenticazione e autorizzazione, crittografia dei dati, limitazione dei privilegi.



# Sicurezza nel software (codice)

- **Gestione dell'output:** È importante validare i dati di output generati dal software per garantire che siano corretti, coerenti e sicuri (sanitizzati). La gestione corretta dell'output nel software contribuisce a garantire l'integrità, la riservatezza e la disponibilità delle informazioni che vengono generate e restituite.

La sicurezza nel software richiede un approccio olistico che coinvolge la progettazione, lo sviluppo, il test e la gestione continua del software. È importante adottare le migliori pratiche di sicurezza e seguire i principi di secure coding per garantire che il software sia robusto, resiliente e protetto da minacce potenziali.



# Test 5

1. Cosa si intende per "buffer overflow"?
  - a) L'inserimento di codice malevolo all'interno del codice sorgente di un'applicazione.
  - b) L'overflow della memoria del buffer di un'applicazione, consentendo a un attaccante di sovrascrivere aree di memoria adiacenti e ottenere il controllo del sistema.
  - c) L'utilizzo di algoritmi di crittografia deboli per proteggere i dati sensibili.
  - d) L'utilizzo di tecniche di hacking per ottenere il controllo completo di un sistema operativo.



# Test 5

2. Quali delle seguenti pratiche contribuiscono a migliorare la sicurezza nel software?
- a) Validazione e sanitizzazione dei dati di input.
  - b) Utilizzo di algoritmi di crittografia robusti.
  - c) Implementazione di controlli di accesso e autorizzazione appropriati.
  - d) Tutte le risposte precedenti.



# Sicurezza Informatica – U2

## IV lezione



# Sicurezza Informatica – U2

## Sommario IV lezione

- Riepilogo comandi Unix/Linux e intro shell
- Esercitazione: Escalation of Privileges
- Sicurezza nei sistemi operativi Unix/Linux (pianificazione, hardening, manutenzione)
- Principi e tecniche di sicurezza dei database





# Comandi Unix/Linux

Unix e Linux sono sistemi operativi basati su riga di comando (CLI - Command Line Interface), che utilizzano una **shell** come **interfaccia utente** per l'interazione con il sistema.

La shell è un programma che permette agli utenti di comunicare con il sistema operativo **attraverso comandi testuali**.

I comandi Unix/Linux sono istruzioni che vengono digitate nella shell per eseguire varie operazioni nel sistema.

Di seguito, un riepilogo introduttivo dei comandi Unix/Linux e una breve introduzione alla shell



# Comandi Unix/Linux

**pwd** (Print Working Directory): Visualizza il percorso completo della directory corrente.

**ls** (List): Elenca i file e le directory nella directory corrente. Alcune opzioni comuni sono:

- l: Visualizza un elenco dettagliato con informazioni sui file (permessi, proprietario, dimensione, data, ecc.)
- a: Mostra tutti i file, inclusi quelli nascosti che iniziano con .

**cd** (Change Directory): Permette di spostarsi tra le directory. Ad esempio, `cd nome_cartella` cambia la directory corrente nella cartella specificata.



# Comandi Unix/Linux

**mkdir** (Make Directory): Crea una nuova directory. Ad esempio, `mkdir nuova_cartella` crea una cartella chiamata "nuova\_cartella".

**rm** (Remove): Rimuove file o directory. Ad esempio, `rm file.txt` rimuove il file "file.txt".

**rmdir** (Remove Directory): Rimuove una directory vuota. Ad esempio, `rmdir vecchia_cartella` rimuove la cartella vuota "vecchia\_cartella".

**cp** (Copy): Copia file o directory. Ad esempio, `cp file.txt backup/` copia il file "file.txt" nella directory "backup".



# Comandi Unix/Linux

**mv** (Move): Sposta o rinomina file o directory. Ad esempio, `mv file.txt nuova_posizione/` sposta il file "file.txt" nella directory "nuova\_posizione", oppure `mv vecchio_nome.txt nuovo_nome.txt` rinomina il file "vecchio\_nome.txt" in "nuovo\_nome.txt".

**touch**: Crea un nuovo file vuoto. Ad esempio, `touch nuovo_file.txt` crea un file chiamato "nuovo\_file.txt".

**cat** (Concatenate): Mostra il contenuto di un file. Ad esempio, `cat file.txt` visualizza il contenuto del file "file.txt".



# Comandi Unix/Linux

**grep** (Global Regular Expression Print): Cerca un pattern (espressione regolare) all'interno di un file o di un output di comando. Ad esempio, `grep parola file.txt` cerca la parola "parola" nel file "file.txt".

**chmod** (Change Mode): Modifica i permessi di accesso di un file o una directory. Ad esempio, `chmod +x script.sh` rende il file "script.sh" eseguibile.

**chown** (Change Owner): Cambia il proprietario di un file o una directory. Ad esempio, `chown nuovo_proprietario file.txt` assegna "nuovo\_proprietario" come proprietario del file "file.txt".



# Comandi Unix/Linux

**ps** (Process Status): Visualizza i processi in esecuzione.

**kill**: Termina un processo. Ad esempio, kill PID termina il processo con il PID (Process ID) specificato.

Inoltre, ci sono diverse shell disponibili in Unix/Linux, tra le più comuni ci sono:

**bash**: Bourne Again SHell, è una delle shell più utilizzate e diffusa di default in molti sistemi Linux.



# Comandi Unix/Linux

sh: Bourne SHell, una delle prime shell di Unix.

zsh: Z SHell, una shell avanzata e potente.

csh: C SHell, una shell con una sintassi simile al linguaggio C.

La shell può essere personalizzata mediante variabili d'ambiente e script di configurazione. Consente di automatizzare task, eseguire script e gestire il sistema in modo efficiente.

Ci sono molti altri comandi e funzionalità avanzate che possono essere esplorati per sfruttare al massimo il potenziale del sistema operativo.



# Comandi Unix/Linux

**man** (Manual): Visualizza il manuale di un comando specifico. Ad esempio, `man ls` mostra il manuale per il comando "ls".

**echo**: Stampa un messaggio sullo schermo o salva un valore in una variabile. Ad esempio, `echo "Ciao Mondo"` stampa "Ciao Mondo" sullo schermo.

**head** e **tail**: Visualizzano le prime o le ultime righe di un file. Ad esempio, `head -n 10 file.txt` mostra le prime 10 righe di "file.txt".

**find**: Cerca file o directory in base a criteri specifici. Ad esempio, `find /path/ -name "*.txt"` cerca tutti i file con estensione ".txt" nella directory specificata.





# Comandi Unix/Linux

**wc** (Word Count): Conta il numero di righe, parole e caratteri in un file. Ad esempio, `wc -l file.txt` conta il numero di righe in "file.txt".

**sort**: Ordina le righe di un file in ordine alfabetico o numerico. Ad esempio, `sort file.txt` ordina le righe di "file.txt".

**uniq**: Rimuove le righe duplicate da un file. Ad esempio, `sort file.txt | uniq` rimuove le righe duplicate da "file.txt".

**tar**: Archivia e comprime file e directory. Ad esempio, `tar -cvzf archive.tar.gz directory/` crea un archivio compresso "archive.tar.gz" contenente i file dalla directory "directory/".



# Comandi Unix/Linux

**grep:** Cerca un pattern (espressione regolare) all'interno di un file o di un output di comando. Ad esempio, `ps aux | grep processo` cerca il processo specificato nell'output del comando "ps aux".

**sed** (Stream Editor): Modifica il contenuto di un file o l'output di un comando utilizzando espressioni regolari. Ad esempio, `sed 's/vecchia/nuova/g' file.txt` sostituisce tutte le occorrenze di "vecchia" con "nuova" nel file "file.txt".

**awk:** Un potente strumento di manipolazione di testo che permette di estrarre e manipolare dati da file o output di comandi. Ad esempio, `awk '{print $1}' file.txt` stampa la prima colonna di "file.txt".



# Comandi Unix/Linux

**df** (Disk Free): Mostra lo spazio disponibile sui dispositivi di archiviazione. Ad esempio, `df -h` mostra lo spazio disponibile in una forma leggibile dall'essere umano.

**du** (Disk Usage): Mostra l'uso dello spazio su disco per file e directory. Ad esempio, `du -h directory/` mostra l'uso dello spazio per la directory specificata.



# Esercizio shell Linux

1. Creare un file di testo di nome "test\_prova1".
2. Selezionare tutte le linee che contengono esattamente la parola "one" (non come sottostringa), ordinarle e farle stampare sullo standard output.
3. Salvare l'output del punto 2 in un file di test.
4. Zippare il file di testo.
5. Cercare parte del contenuto del file di output in tutto il S.O.



# Escalation of privileges

La "escalation of privileges" o "elevation of privileges" è un processo attraverso il quale un utente o un processo tenta di ottenere privilegi di amministratore (root) o di un utente con livelli di autorizzazione superiori a quelli assegnati di default. Questo tipo di attacco è spesso utilizzato dagli hacker per ottenere un maggiore accesso e controllo su un sistema.

È importante sottolineare che l'escalation di privilegi è un'azione illegale e può essere perseguibile penalmente. Quindi, la seguente dimostrazione è fornita solo a scopo educativo per mostrare come avviene l'escalation di privilegi e per evidenziare l'importanza di proteggere correttamente i sistemi.



# Escalation of privileges

Supponiamo di avere un utente standard "utente1" con accesso limitato e vogliamo dimostrare come "utente1" potrebbe tentare di ottenere l'accesso root utilizzando un'escalation di privilegi.

1. Verifica dei privilegi dell'utente corrente:

```
> id
```

```
uid=1000(utente1) gid=1000(utente1)
```

```
groups=1000(utente1),4(adm),24(cdrom),27(sudo),30(dip),4  
6(plugdev),116(lpadmin),126(sambashare)
```

2. Verifica dei file eseguibili con permessi elevati (suid/sgid):

```
> find / -type f \( -perm -4000 -o -perm -2000 \) -exec ls -l {} \;
```



# Escalation of privileges

3. Utilizzo dell'eseguibile "ex\_elevate" con permessi elevati per ottenere l'accesso root  
    > ./ex\_elevate

Nell'esempio sopra, l'eseguibile "ex\_elevate" è configurato con il bit suid (setuid), il che significa che quando viene eseguito, ottiene temporaneamente i privilegi dell'utente proprietario del file (in questo caso "root").

Pertanto, quando "utente1" esegue "ex\_elevate", ottiene l'accesso temporaneo come utente root.



# Escalation of privileges

Approfondimento:

Come assegnare i permessi di esecuzione e impostare il bit suid a un file?

Ipotizziamo che il file si chiama “elevate”

- > `chmod +x elevate`
- > `sudo chown root:root elevate`
- > `sudo chmod +s elevate`





# Sicurezza nei sistemi operativi Unix/Linux

## ❖ Pianificazione della sicurezza:

- Valutazione dei requisiti di sicurezza: Effettuare un'analisi approfondita delle esigenze di sicurezza del sistema, tenendo conto dei dati sensibili, delle minacce potenziali e dei requisiti normativi o aziendali. Questo aiuterà a identificare le aree critiche che richiedono una protezione speciale.
- Politiche di sicurezza: Definire politiche di sicurezza dettagliate che includano regole per l'accesso, l'autenticazione, la crittografia, la gestione delle password, il controllo degli accessi, ecc. Queste politiche dovrebbero essere comunicate e applicate a tutti gli utenti e gli amministratori del sistema.



# Sicurezza nei sistemi operativi Unix/Linux

- Assegnazione dei ruoli e delle responsabilità: Definire chiaramente i ruoli e le responsabilità di ogni utente, amministratore di sistema e responsabile della sicurezza. Limitare i privilegi amministrativi solo agli utenti che ne hanno effettivamente bisogno per svolgere il proprio lavoro.



## ❖ Hardening del sistema:

- Aggiornamento del sistema: Mantenere costantemente aggiornato il sistema operativo e il software con le ultime patch di sicurezza. Impostare l'aggiornamento automatico per semplificare il processo di applicazione delle patch.

# Sicurezza nei sistemi operativi Unix/Linux

- Configurazione sicura: Disabilitare o rimuovere i servizi di sistema non necessari. Configurare correttamente le impostazioni di sicurezza, come disattivare il login remoto con password per l'account root, limitare l'uso di privilegi di root solo quando necessario, ecc.
- Firewall: Configurare un firewall per filtrare il traffico di rete, consentendo solo le connessioni essenziali alle porte necessarie. Questo aiuterà a prevenire gli attacchi dall'esterno.
- Controllo degli accessi: Impostare attentamente le autorizzazioni dei file e delle directory per garantire che solo gli utenti autorizzati possano accedere ai dati e alle risorse critiche. Utilizzare gli attributi di sicurezza come SELinux o AppArmor, se disponibili, per limitare ulteriormente i privilegi.



# Sicurezza nei sistemi operativi Unix/Linux

- Limitazione dei privilegi: Ridurre i privilegi degli utenti limitando l'accesso a funzioni e comandi critici. Utilizzare sudo per consentire agli utenti autorizzati di eseguire specifici comandi con privilegi di amministratore, invece di concedere accesso completo all'account di root.



## ❖ Manutenzione continua:

- Monitoraggio: Implementare un sistema di monitoraggio che consenta di rilevare attività sospette o intrusioni nel sistema. Utilizzare strumenti come log di sicurezza, sistemi di rilevamento delle intrusioni (IDS), sistemi di analisi dei log, ecc.

# Sicurezza nei sistemi operativi Unix/Linux

- Logging: Abilitare il logging dettagliato di eventi di sicurezza, inclusi login, accessi a file, operazioni di amministrazione, ecc. I log devono essere protetti da modifiche non autorizzate e archiviati in modo sicuro per l'analisi successiva.
- Pianificazione delle patch: Creare un piano di aggiornamento regolare per applicare tempestivamente le patch di sicurezza e le correzioni di vulnerabilità. Monitorare le notifiche di sicurezza per essere informati sugli aggiornamenti critici.
- Backup: Eseguire regolarmente backup completi e incrementali dei dati importanti. Assicurarsi che i backup siano memorizzati in una posizione sicura, preferibilmente in un dispositivo separato, per evitare la perdita dei dati in caso di attacco o guasto del sistema.



# Sicurezza nei sistemi operativi Unix/Linux

La sicurezza nei sistemi operativi Unix/Linux richiede un approccio olistico, combinando pianificazione, configurazione sicura, monitoraggio attivo e pratica costante di buone pratiche di sicurezza. Inoltre, è fondamentale educare gli utenti riguardo alle migliori pratiche di sicurezza e alle minacce informatiche, in modo che possano contribuire attivamente alla sicurezza del sistema.

**Approfondimento:** L'hardening, nell'ambito della sicurezza informatica, è il processo di rafforzamento e protezione di un sistema informatico riducendo le sue vulnerabilità e aumentando la sua resistenza agli attacchi. L'obiettivo dell'hardening è rendere il sistema più sicuro e meno suscettibile agli exploit o alle intrusioni.



# Principi e tecniche di sicurezza dei database

I principi e le tecniche di sicurezza dei database sono fondamentali per proteggere i dati sensibili e critici memorizzati nei sistemi di gestione dei database (DBMS).

La sicurezza dei database mira a prevenire accessi non autorizzati, garantire la riservatezza, l'integrità e la disponibilità dei dati, oltre a proteggere i sistemi da attacchi informatici e minacce interne.

Di seguito sono elencati alcuni principi e tecniche comuni di sicurezza dei database



# Principi e tecniche di sicurezza dei database

## Accesso autorizzato:

- Implementare un sistema di autenticazione robusto: Utilizzare metodi di autenticazione sicuri, come l'autenticazione a più fattori (MFA), per verificare l'identità degli utenti che cercano di accedere al database.
- Gestire le credenziali in modo sicuro: Assicurarsi che le password siano memorizzate in modo crittografato e che non siano facilmente accessibili da utenti non autorizzati o da applicazioni.





# Principi e tecniche di sicurezza dei database

## Gestione delle autorizzazioni:

- Assegnare i privilegi in modo oculato: Concedere i privilegi solo agli utenti che ne hanno effettivamente bisogno per svolgere le proprie attività, evitando di concedere accessi indiscriminati.
- Utilizzo dei ruoli: Creare ruoli e assegnare i privilegi ai ruoli, quindi assegnare i ruoli agli utenti. Ciò semplifica la gestione delle autorizzazioni e rende più facile il controllo dell'accesso.



# Principi e tecniche di sicurezza dei database

## Crittografia dei dati:

- Utilizzo della crittografia a livello di colonna: Crittografare specifiche colonne contenenti dati sensibili, in modo che solo gli utenti autorizzati possano leggere il contenuto decriptato.
- Crittografia end-to-end: Proteggere i dati crittografandoli quando vengono trasmessi sulla rete (in transito) e quando vengono memorizzati nel database (a riposo).



# Principi e tecniche di sicurezza dei database

## Auditing e logging:

- Implementare il logging dettagliato: Registrare tutte le attività significative del database, inclusi gli accessi, le modifiche ai dati e le operazioni di amministrazione.
- Monitoraggio attivo: Monitorare costantemente i log per individuare attività sospette o tentativi di accesso non autorizzato.



# Principi e tecniche di sicurezza dei database

## Backup e recovery:

- Pianificare regolari backup: Eseguire backup completi e incrementali del database in modo che i dati possano essere recuperati in caso di perdita o danneggiamento.
- Archiviazione sicura: Conservare i backup in luoghi sicuri e separati dal sistema di produzione per proteggerli da accessi non autorizzati o da eventi catastrofici.



# Principi e tecniche di sicurezza dei database

## Gestione delle vulnerabilità:

- Applicazione tempestiva delle patch: Mantenere il sistema del database e il software sempre aggiornati con gli ultimi patch di sicurezza per proteggere il sistema da vulnerabilità conosciute.



## Prevenzione delle injection:

- Utilizzo di prepared statements: Utilizzare prepared statements o stored procedure nei linguaggi di query per evitare attacchi di SQL injection.
- Validazione e sanificazione dei dati: Verificare e sanificare correttamente i dati di input prima di utilizzarli in query o comandi del database.

# Principi e tecniche di sicurezza dei database

## Sicurezza fisica e accesso fisico:

- Accesso controllato alla struttura fisica: Limitare l'accesso fisico ai server e ai dispositivi di archiviazione dei dati solo al personale autorizzato.



## Controllo delle transazioni:

- Implementare transazioni: Utilizzare le transazioni per garantire l'integrità dei dati e per gestire operazioni complesse in modo atomico, coerente, isolato e durevole (ACID).

# Principi e tecniche di sicurezza dei database

## **Separazione dei doveri:**

- Dividere i compiti tra diverse figure: Assegnare responsabilità specifiche a persone diverse per ridurre il rischio di abusi di potere o di errori accidentali.



## **Gestione delle sessioni:**

- Imporre politiche di gestione delle sessioni: Terminare automaticamente le sessioni inattive dopo un certo periodo di tempo per prevenire session hijacking.

# Principi e tecniche di sicurezza dei database

## Test di sicurezza:

- Penetration test e vulnerability assessment: Eseguire regolari test di sicurezza per identificare e risolvere le vulnerabilità prima che vengano sfruttate dagli attaccanti.



Implementando questi principi e utilizzando queste tecniche, le organizzazioni possono migliorare significativamente la sicurezza dei loro database e proteggere i dati sensibili dai rischi di violazione o perdita. La sicurezza dei database è una pratica continua che richiede vigilanza costante e adattabilità per far fronte alle nuove minacce e alle nuove tecnologie.



# Sicurezza Informatica – U2

## V lezione



# Sicurezza Informatica – U2

## Sommario IV lezione

- Sicurezza nei container:
  - Docker
  - Kubernetes
- Sicurezza nel dispiegamento di software su cloud:
  - software e banche di dati; Open Source, GNU General Public License (GPL), licenze Creative Commons)



# Container

## Cosa sono i Container:

I container sono una *tecnologia di virtualizzazione* leggera che consente di eseguire applicazioni e i loro dipendenze in un *ambiente isolato* chiamato "container".

I container sono diventati estremamente popolari negli ultimi anni per la distribuzione di applicazioni, lo sviluppo software e la gestione delle risorse informatiche. Seguono le caratteristiche chiave dei container:

- **Isolamento:** I container forniscono un alto grado di isolamento tra l'applicazione e l'host in cui vengono eseguiti. Ciò significa che un'applicazione in un container non interferisce con altre applicazioni o con il sistema operativo sottostante.



# Container

- **Portabilità:** I container sono altamente portabili. Un container può essere creato e testato su un ambiente di sviluppo e quindi eseguito in modo coerente su un ambiente di produzione, *indipendentemente dal sistema operativo o dall'infrastruttura sottostante*.
- **Leggerezza:** I container sono leggeri rispetto alle macchine virtuali (VM). Condividono il kernel del sistema operativo dell'host e condividono risorse con altri container, il che li rende molto efficienti dal punto di vista delle risorse.



# Container

- **Riproducibilità:** I container sono basati su immagini, che rappresentano un pacchetto completo di un'applicazione e delle sue dipendenze. Queste immagini possono essere create, distribuite e replicare in modo coerente.
- **Orchestrazione:** Per gestire un gran numero di container, spesso si fa uso di strumenti di orchestrazione come Kubernetes, Docker Swarm o OpenShift. Questi strumenti semplificano la distribuzione, la scalabilità e la gestione dei container su cluster di server.



# Container

- **DevOps** e Continuous Integration/Continuous Deployment (CI/CD): I container sono spesso utilizzati nell'ambito delle pratiche DevOps e CI/CD per automatizzare il processo di sviluppo, test e distribuzione delle applicazioni. Ciò consente un rilascio rapido e affidabile del software.

I container sono diventati uno strumento essenziale per gli sviluppatori e gli amministratori di sistema per semplificare la distribuzione delle applicazioni, migliorare la gestione delle risorse e rendere più efficienti le operazioni IT.

*Docker* è uno dei sistemi di gestione dei container più noti ed è spesso utilizzato come termine generico per riferirsi ai container, anche se esistono altre tecnologie.



# Container

Ecco alcuni dei container principali e degli strumenti associati:

- **Docker:** Docker è stato uno dei pionieri nella tecnologia dei container ed è noto per la sua facilità d'uso. Docker fornisce un'ampia gamma di strumenti per la creazione, la gestione e l'esecuzione di container. Docker Hub è un registro pubblico di immagini Docker pronte all'uso.
- **Kubernetes:** Kubernetes (spesso abbreviato come "K8s") è un sistema open-source per l'orchestrazione di container. È utilizzato per gestire e scalare applicazioni containerizzate su cluster di server. Kubernetes offre un alto livello di automazione e scalabilità.



# Container

- **Containerd:** Containerd è un runtime di container open-source che gestisce l'esecuzione dei container stessi. È un componente fondamentale di Docker e di altri sistemi di containerizzazione.
- **rkt (Rocket):** rkt è un runtime di container open-source sviluppato da CoreOS. È noto per la sua attenzione alla sicurezza e al principio del "minimalismo". Sebbene non sia così diffuso come Docker, è utilizzato in alcuni scenari specifici.
- **Podman:** Podman è un altro strumento di containerizzazione open-source, progettato per essere una alternativa compatibile con Docker. Si concentra sulla compatibilità con Docker e sulla sicurezza.





# Container

- **OpenShift:** OpenShift è una piattaforma di containerizzazione e orchestrazione basata su Kubernetes, sviluppata da Red Hat. Offre funzionalità aggiuntive per la gestione, la sicurezza e lo sviluppo delle applicazioni containerizzate.
- **Docker Swarm:** Docker Swarm è uno strumento di orchestratura dei container integrato direttamente in Docker. È noto per la sua facilità d'uso e viene utilizzato per gestire cluster di container Docker.
- **CRI-O:** CRI-O è un runtime di container dedicato all'integrazione con Kubernetes. È progettato per eseguire container all'interno di cluster Kubernetes.



# Docker VS Kubernetes

| Caratteristica         | Docker                                                                                                              | Kubernetes                                                                                                              |
|------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Tipologia              | Piattaforma di containerizzazione                                                                                   | Sistema di orchestrazione dei container                                                                                 |
| Scopo principale       | Creare, distribuire e gestire container                                                                             | Orchestrare e gestire container su cluster                                                                              |
| Livello di astrazione  | Livello di container                                                                                                | Livello di orchestratore                                                                                                |
| Isolamento             | Offre isolamento leggero tra container                                                                              | Fornisce isolamento avanzato e orchestrazione tra container                                                             |
| Gestione delle risorse | Gestisce risorse per singoli container                                                                              | Gestisce risorse e scalabilità per cluster di container                                                                 |
| Networking             | Offre funzionalità di rete tra container su un singolo host                                                         | Gestisce la rete tra i container su host diversi all'interno di un cluster                                              |
| Orchestrazione         | Fornisce strumenti di base per la gestione dei container, ma richiede strumenti di terze parti per l'orchestrazione | Offre funzionalità avanzate di orchestrazione, inclusi servizi di bilanciamento del carico, autoscaling e distribuzione |
| Load Balancing         | Non ha funzionalità di bilanciamento del carico incorporato                                                         | Fornisce bilanciamento del carico tra i container                                                                       |

# Docker VS Kubernetes

|                              |                                                                                      |                                                                                          |
|------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>High Availability</b>     | Richiede configurazioni aggiuntive per l'alta disponibilità                          | Fornisce un alto livello di disponibilità tramite replicazione dei container             |
| <b>Scalabilità</b>           | La scalabilità dei container è limitata all'host in cui sono in esecuzione           | Permette la scalabilità orizzontale su più host nel cluster                              |
| <b>Deployment</b>            | I container devono essere distribuiti manualmente o tramite strumenti di terze parti | Fornisce funzionalità avanzate di distribuzione e aggiornamento dei container            |
| <b>Stato e Gestione</b>      | Limitato controllo dello stato del container                                         | Fornisce un maggiore controllo sullo stato dei container e dei servizi                   |
| <b>Comunità e Ecosistema</b> | Ampio ecosistema di contenitori con una vasta adozione                               | Ecosistema dedicato all'orchestrazione dei container                                     |
| <b>Utilizzo tipico</b>       | Sviluppo, test e distribuzione di container su un singolo host                       | Orchestrazione e gestione di applicazioni containerizzate complesse su cluster di server |



In sintesi, Docker è una piattaforma per la containerizzazione, mentre Kubernetes è un sistema di orchestrazione per la gestione di container su cluster.

# Sicurezza in Docker

La sicurezza in Docker è una preoccupazione fondamentale, poiché Docker e i container in generale condividono il kernel del sistema operativo host e possono rappresentare una superficie di attacco potenziale se non gestiti correttamente. Seguono alcune delle principali considerazioni e pratiche per la gestione della sicurezza in Docker:

- **Immagini sicure:** Utilizzare immagini ufficiali o immagini da fonti fidate come Docker Hub. Ispezionare e verificare le immagini di base per garantire che siano aggiornate e prive di vulnerabilità conosciute.



# Sicurezza in Docker

- **Aggiornamenti e patch:**

Mantenere aggiornato Docker e il sistema operativo dell'host con gli ultimi aggiornamenti di sicurezza per proteggere da vulnerabilità note.

Implementare un processo di monitoraggio delle vulnerabilità per le immagini dei container in uso e applicare patch o aggiornamenti quando necessario.

- **Configurazione sicura:**

Limitare i privilegi: Eseguire i container con i privilegi minimi necessari. Utilizzare il principio del "minimo privilegio".

Disabilitare funzionalità non necessarie nei container, ad esempio le capacità del kernel non utilizzate.



# Sicurezza in Docker

- **Controllo degli accessi:**

Utilizzare i meccanismi di controllo degli accessi di Docker per definire chi può accedere e quali risorse possono essere utilizzate.

Implementare il principio del "principio del privilegio minimo" per concedere solo i privilegi necessari ai container e agli utenti.



- **Rete sicura:**

Utilizzare le reti Docker isolate o reti personalizzate per separare i container in ambienti di rete virtuale.

Limitare l'esposizione delle porte solo ai servizi necessari e utilizzare il mapping delle porte in modo sicuro.

# Sicurezza in Docker

- **Gestione delle credenziali:**

Gestire in modo sicuro le credenziali e le variabili d'ambiente sensibili, ad esempio le password dei database, utilizzando strumenti come Docker Secrets o Docker Configs.

- **Scansione delle vulnerabilità:**

Utilizzare strumenti di scansione delle vulnerabilità per eseguire verifiche regolari sulle immagini dei container per rilevare e correggere le vulnerabilità note.



# Sicurezza in Docker

- **Monitoraggio e logging:**

Configurare il monitoraggio per i container per rilevare comportamenti anomali.

Implementare il logging dettagliato per catturare eventi e attività rilevanti nei container.

- **Limitazione delle risorse:**

Impostare limiti di utilizzo delle risorse, come CPU e memoria, per impedire a un container di monopolizzare le risorse dell'host.

- **Auditing:**

Abilitare l'auditing dei container per raccogliere registrazioni dettagliate delle attività dei container e delle azioni degli utenti.





# Sicurezza in Docker

- **Backup e ripristino:**  
Eseguire regolari backup dei dati dei container per garantire la disponibilità e la ripristinabilità dei dati in caso di problemi.
- **Orchestrazione sicura:**  
Se si utilizza un'orchestrazione come Kubernetes o Docker Swarm, assicurarsi di implementare le pratiche di sicurezza specifiche per l'orchestrazione.



# Sicurezza in Kubernetes

La sicurezza in Kubernetes è di fondamentale importanza, dato che Kubernetes è utilizzato per orchestrare e gestire container in ambienti complessi e spesso condivisi.

Come si gestisce in sicurezza in Kubernetes?

- **Aggiornamenti e patch:**  
Mantenere Kubernetes e i componenti del cluster (come kubelet e kube-proxy) aggiornati con gli ultimi aggiornamenti di sicurezza.



# Sicurezza in Kubernetes

## **Accesso e autenticazione:**

Utilizzare un'infrastruttura di autenticazione forte come l'autenticazione a più fattori (MFA) per gli accessi al cluster. Limitare l'accesso solo agli utenti e alle applicazioni autorizzati, utilizzando autenticazione basata su token, certificati e ruoli di Kubernetes (RBAC).



## **Controllo degli accessi:**

Utilizzare Role-Based Access Control (RBAC) per definire e limitare i permessi degli utenti e dei servizi all'interno del cluster.

Implementare il principio del "principio del privilegio minimo", assegnando solo i permessi necessari a ciascun utente o servizio.

# Sicurezza in Kubernetes

## **Rete sicura:**

Utilizzare Network Policies per definire regole di sicurezza per il traffico di rete tra i pod. Limitare le comunicazioni solo ai servizi e ai pod autorizzati.

Implementare crittografia per la comunicazione tra i componenti di Kubernetes (ad esempio, etcd) e tra i pod utilizzando servizi come HTTPS.



## **Gestione delle credenziali:**

Utilizzare Kubernetes Secrets o strumenti esterni come Vault per gestire in modo sicuro le credenziali e le variabili d'ambiente sensibili.

# Sicurezza in Kubernetes

## **Scansione delle vulnerabilità:**

Eseguire regolari scansioni delle vulnerabilità nelle immagini dei container per rilevare e correggere vulnerabilità conosciute.

Utilizzare immagini di base sicure e affidabili.

## **Monitoraggio e logging:**

Configurare sistemi di monitoraggio per rilevare comportamenti anomali e intrusioni nel cluster Kubernetes. Implementare il logging dettagliato per catturare eventi e attività rilevanti nei container e nei componenti del cluster.



# Sicurezza in Kubernetes

## **Politiche di sicurezza:**

Implementare politiche di sicurezza centralizzate utilizzando strumenti come Open Policy Agent (OPA) per garantire che tutte le risorse di Kubernetes rispettino le regole di sicurezza.

## **Limitazione delle risorse:**

Impostare limiti di utilizzo delle risorse (CPU e memoria) per i container e i pod per evitare l'abuso delle risorse del cluster.

## **Auditing:**

Abilitare l'auditing in Kubernetes per raccogliere registrazioni dettagliate delle attività all'interno del cluster.



# Sicurezza in Kubernetes

## **Backup e ripristino:**

Eseguire regolari backup dello stato del cluster, compreso il database etcd di Kubernetes, per garantire la ripristinabilità in caso di guasti.

## **Hardening dell'host:**

Assicurarsi che gli host su cui viene eseguito Kubernetes siano correttamente hardenizzati, con configurazioni di sistema sicure e aggiornamenti regolari.

## **Strumenti di sicurezza:**

Utilizzare strumenti di sicurezza specifici per Kubernetes, come kube-bench e kube-hunter, per valutare la sicurezza del cluster.



# Sicurezza in Kubernetes

## **Gestione delle identità delle applicazioni:**

Implementare un sistema di gestione delle identità delle applicazioni (IAM) per le applicazioni e i servizi all'interno del cluster.

## **N.B.**

Inoltre, la sicurezza in Kubernetes è un processo in continuo sviluppo a causa delle nuove minacce e delle evoluzioni nella tecnologia.





# Sicurezza nel dispiegamento software in Cloud

La sicurezza nel dispiegamento di software su cloud è una preoccupazione fondamentale quando si tratta di gestire software e banche dati, indipendentemente dal fatto che si tratti di software open-source o di dati soggetti a licenze Creative Commons.

In generale, a prescindere dalla tipologia di licenza, è fondamentale **proteggere i dati, rispettare le normative** (come GDPR), **implementare** strumenti di **backup e ripristino**



# Sicurezza nel dispiegamento software in Cloud

## Software Open-Source

Il software open-source è accessibile al pubblico e può essere utilizzato, modificato e distribuito liberamente.

La sicurezza nei progetti open-source è responsabilità della comunità di sviluppatori e degli utenti. È importante utilizzare software open-source da fonti affidabili e mantenerlo aggiornato.

Verificare periodicamente se ci sono vulnerabilità di sicurezza note nei progetti open-source utilizzati e applicare patch o aggiornamenti quando necessario.



# Sicurezza nel dispiegamento software in Cloud

## GNU General Public License (GPL)

La GPL è una delle licenze più comuni per il software open-source. Impone che il codice sorgente del software sia disponibile e modificabile da chiunque.

Quando si utilizza software con licenza GPL, è necessario rispettarne i termini, che comprendono la condivisione delle modifiche apportate al codice sorgente.



# Sicurezza nel dispiegamento software in Cloud

## Licenze Creative Commons

Le licenze Creative Commons consentono ai creatori di opere (come testi, immagini o altro) di specificare le condizioni di utilizzo da parte di altri.

È importante rispettare le condizioni specificate nelle licenze Creative Commons quando si utilizzano opere protette da tali licenze.

Queste condizioni possono variare da un'opera all'altra.



# Grazie per l'attenzione

**Docente:**

Dott. Ing. Giuseppe Placanica

[giuseppe.placanica@silicondev.com](mailto:giuseppe.placanica@silicondev.com)

